

Advanced Natural Language Processing

Comprehensive Final Examination Study Notes

Course Notes

1 Part I: Recurrent Models

1.1 The Simple Recurrent Neural Network (RNN)

What it is. A neural network with a loop: it processes a sequence one token at a time and carries a hidden state vector from each step into the next, so that the state acts as a summary of everything seen so far.

The idea, plainly. Imagine reading a sentence with a single sticky note in your hand. At each word you (i) read the word, (ii) read your sticky note, (iii) write a new sticky note that combines the two, and (iv) throw the old one away. The sticky note is the hidden state $h^{(t)}$. Because you rewrite it in exactly the same way at every word, you can read a sentence of any length with a fixed amount of machinery.

Mechanics. With input embedding $x^{(t)}$ and hidden state $h^{(t)}$:

$$h^{(t)} = \tanh(Uh^{(t-1)} + Wx^{(t)}), \quad o^{(t)} = \text{softmax}(Vh^{(t)}).$$

Three observations that the exam likes:

- x and h change at every time step; the parameters U , W , V do **not**. They are shared across all positions.
- U mixes in history, W mixes in the present word, V reads out a prediction.
- $h^{(0)}$ is usually the zero vector.

Problem it solves. Variable-length input and order-sensitivity, using a constant number of parameters. Weight sharing means a model trained on 10-word sentences can process a 40-word sentence.

Shapes of RNN problems. The same cell supports several task shapes (Karpathy's taxonomy):

Shape	Example
one-to-one	Ordinary classification (no recurrence really needed).
one-to-many	Image captioning: one image in, a sentence out.
many-to-one	Sentiment analysis: the last hidden state represents the whole sentence.
many-to-many (aligned)	POS tagging / NER: one label per token.
many-to-many (unaligned)	Machine translation: read all, then write all.

Advantages.

- Handles arbitrary length with fixed parameter count.
- Parameter sharing gives strong generalisation across positions.
- Small memory footprint; state is one vector.
- Naturally streaming/online — it can process a token the moment it arrives.

Disadvantages.

- **Sequential by construction**, so it cannot be parallelised along time. Step t needs $h^{(t-1)}$.
- **Vanishing (and exploding) gradients** during backpropagation through time.
- **It can only see the past**, not the future context of a token.
- **It must forget the same amount of history at every step**, because the update is a single fixed tanh transformation — there is no mechanism to say “this token matters, keep it for 50 steps”.

What it cannot solve. Long-term dependencies. Theoretically a simple RNN is capable of representing them — you could hand-pick weights that do it — but in practice gradient descent does not find those weights. This gap between representational capacity and learnability is a favourite exam distinction.

What it gave rise to. The vanishing gradient problem, which motivated gating (LSTM, GRU); the past-only limitation, which motivated bidirectional RNNs; the sequential bottleneck, which eventually motivated the Transformer.

Use it when. Short sequences, streaming/low-latency settings, very small compute or memory budgets, or as a cheap baseline.

Do not use it when. Dependencies span more than roughly 10–20 tokens, you have GPUs and want to exploit them, or you need state-of-the-art accuracy.

Counterfactual. *What if the weights were not shared across time steps?* You would need a separate U, W, V per position. The parameter count would grow with sequence length, the model could never process a sequence longer than the longest one it was built for, and a pattern learned at position 3 would not transfer to position 30. Weight sharing is not an optimisation detail — it is the entire reason an RNN generalises over sequences.

1.2 How the Input Is Represented, and How Big Each Matrix Is

The input layer size is set by the word representation, not the sentence length. This trips people up. In a feed-forward network on tabular data, the input layer has one neuron per column. In an RNN, the number of words decides the number of *time steps*; the input layer size is decided by how a single word is written down as numbers:

- **One-hot encoding:** a vocabulary of 30 words gives a 30-dimensional input vector — all zeros with a single 1 in the slot for that word. So 30 input neurons.
- **Word embeddings:** each word is squashed into a dense vector of, say, 300 numbers. So 300 input neurons. This is what modern systems use.

A 5-word sentence and a 50-word sentence both use the same input layer; the longer one just runs the loop more times.

The three matrices, in plain words.

Matrix	Its job, in one question	Connects
W	“How do I read a new word?”	input $\rightarrow$ hidden state
U	“How much do I trust my old notes?”	old hidden state $\rightarrow$ new hidden state
V	“How do I turn my notes into an answer?”	hidden state $\rightarrow$ output

They are three separate matrices because they are three genuinely different jobs, each needing its own learned behaviour. Squashing them into one matrix would force the model to read a word and recall its memory using the same weights.

Shapes, with concrete numbers. Take vocabulary = 30, hidden size = 5, output classes = 3. The matrix-multiplication rule is that $[a \times b] \cdot [b \times 1] = [a \times 1]$, so:

- $Wx^{(t)}$ must give $[5 \times 1]$ from a $[30 \times 1]$ input, so W is $[5 \times 30]$.
- $Uh^{(t-1)}$ must give $[5 \times 1]$ from a $[5 \times 1]$ state, so U is $[5 \times 5]$.
- $Vh^{(t)}$ must give $[3 \times 1]$ from a $[5 \times 1]$ state, so V is $[3 \times 5]$.

Total: $150 + 25 + 15 = 190$ parameters. Remember this number — the LSTM comparison below uses it.

Why U is always square. U takes the hidden state in and must produce something the same size, because that output becomes the next hidden state and will be fed through U again. Any other shape breaks the loop after one step.

The hidden state is not a learned parameter. h is *recomputed* at every step of every sequence. It starts at the zero vector (empty memory), is rebuilt at each step, and is thrown away when the sequence ends. What gets learned is W , U , V — never h .

When does the output layer fire? It depends on the task shape. In a many-to-one task (sentiment), steps 1–4 only update memory and the output layer stays silent; the prediction is made at step 5. In a many-to-many task (POS tagging), an output is produced at every step.

Why weights are updated after the *whole* sequence, not after each word. Two reasons, and they reinforce each other. First, W , U , V are shared across all steps, so the same weight is partly responsible for the error at step 1 and the error at step 5. Second, the loss at step 3 was partly caused by weights acting at steps 1 and 2, because h_1 flows into h_2 flows into h_3 . You cannot assign blame fairly without the total loss over the full sequence. Contrast a feed-forward network on tabular data, where rows are independent and weights are not shared, so an update after every row is perfectly sensible.

1.3 Worked Example: A Vanilla RNN Tagging “the hair dryer is great”

Nothing here is new mathematics — it is the same two equations, done with actual numbers so the shapes stop being abstract.

Setup. Five words, five POS classes. One-hot inputs of size 5, hidden size 5, output size 5. Tag indices: 0 =DET, 1 =NOUN, 2 =VERB, 3 =ADV, 4 =ADJ. True labels: the→0, hair→1, dryer→1, is→2, great→4. To keep the arithmetic clean, set $W = \mathbf{I}_5$, $U = 0.5 \mathbf{I}_5$, $V = 0.5 \mathbf{I}_5$, $h^{(0)} = \mathbf{0}$. Because all three are diagonal, multiplying is just element-wise scaling.

Step 1 — “the”.

$$\begin{aligned} Uh^{(0)} + Wx^{(1)} &= [0, 0, 0, 0, 0] + [1, 0, 0, 0, 0] = [1, 0, 0, 0, 0] \\ h^{(1)} &= \tanh([1, 0, 0, 0, 0]) = [0.7616, 0, 0, 0, 0] \\ o^{(1)} &= \text{softmax}([0.3808, 0, 0, 0, 0]) = [0.2679, 0.1830, 0.1830, 0.1830, 0.1830] \\ L^{(1)} &= -\log(0.2679) = 1.3173 \quad \checkmark \text{ DET is the argmax} \end{aligned}$$

Step 2 — “hair”. Now the old state contributes for the first time:

$$\begin{aligned} Uh^{(1)} + Wx^{(2)} &= [0.3808, 0, 0, 0, 0] + [0, 1, 0, 0, 0] = [0.3808, 1, 0, 0, 0] \\ h^{(2)} &= [0.3634, 0.7616, 0, 0, 0], \quad o^{(2)} = [0.2118, 0.2584, 0.1766, 0.1766, 0.1766] \\ L^{(2)} &= -\log(0.2584) = 1.3531 \quad \checkmark \text{ NOUN} \end{aligned}$$

Notice the memory effect: slot 1 (“the”) has faded from 0.7616 to 0.3634 without anyone deciding to forget it. That automatic, uncontrolled decay is precisely what the LSTM exists to replace.

Steps 3–5, in brief.

$$\begin{aligned} h^{(3)} &= [0.1797, 0.3634, 0.7616, 0, 0], & L^{(3)} &= 1.5687 \times (\text{VERB, true NOUN}) \\ h^{(4)} &= [0.0896, 0.1797, 0.3634, 0.7616, 0], & L^{(4)} &= 1.5766 \times (\text{ADV, true VERB}) \\ h^{(5)} &= [0.0448, 0.0896, 0.1797, 0.3634, 0.7616], & L^{(5)} &= 1.3814 \checkmark \text{ ADJ} \\ L_{\text{total}} &= \frac{1}{5}(1.3173 + 1.3531 + 1.5687 + 1.5766 + 1.3814) = 1.4394. \end{aligned}$$

The backward pass, in outline. For softmax with cross-entropy the gradient at the output is simply $\delta_{\text{out}}^{(t)} = o^{(t)} - y^{(t)}$ — prediction minus truth. For example at step 1, $\delta_{\text{out}}^{(1)} = [-0.7321, 0.1830, 0.1830, 0.1830, 0.1830]$: negative on the correct class (push it up), positive elsewhere (push them down). Then, walking backwards from $t = 5$ to $t = 1$:

$$\begin{aligned} \delta_h^{(t)} &= V^\top \delta_{\text{out}}^{(t)} / T + \delta_{h,\text{next}} \\ \delta_{\text{pre}}^{(t)} &= \delta_h^{(t)} \odot \left(1 - [h^{(t)}]^2\right) && \leftarrow \tanh \text{ derivative} \\ \nabla W &+= \delta_{\text{pre}}^{(t)} \otimes x^{(t)}, & \nabla U &+= \delta_{\text{pre}}^{(t)} \otimes h^{(t-1)} \\ \delta_{h,\text{next}} &\leftarrow U^\top \delta_{\text{pre}}^{(t)} \end{aligned}$$

Two things worth memorising from this. First, the gradient of a shared weight is the *sum* of its contributions from every time step — that is what “shared weights” means at update time. Second, the last column of ∇U comes out all zeros, because $h^{(0)} = \mathbf{0}$ and so nothing flows back past the start of the sequence.

1.4 Backpropagation Through Time and the Vanishing Gradient

What it is. BPTT is ordinary backpropagation applied to the RNN after “unrolling” it: the loop is expanded into a deep feed-forward network with one layer per time step, all layers sharing the same weights.

The idea, plainly. A 50-word sentence turns the RNN into a 50-layer network. Gradients must be carried back through all 50 layers by repeated multiplication.

Mechanics, and why it fails. The gradient of the loss at step T with respect to the state at step t contains a product of $T - t$ Jacobians:

$$\frac{\partial h^{(T)}}{\partial h^{(t)}} = \prod_{k=t+1}^T \frac{\partial h^{(k)}}{\partial h^{(k-1)}}.$$

Each factor involves U and the derivative of $\tanh$ (which is at most 1, and usually much less). Multiplying many numbers smaller than one gives a compounding shrinkage:

- If the factors are < 1 , the gradient decays roughly geometrically. After 30–50 steps it is numerically zero: the early time steps receive no learning signal. This is the **vanishing gradient problem**.
- If the factors are > 1 , the gradient blows up: the **exploding gradient problem**. This one is easy to patch with gradient clipping. Vanishing is the hard one, because you cannot “unclip” a zero.

Why this is a modelling problem, not a numerical nuisance. A zero gradient at early time steps means the model is literally unable to learn that the word “France” 25 tokens ago should influence the current prediction. The failure is in learning long-term dependencies, exactly the property we needed.

Counterfactual. *What if the state update were additive instead of multiplicative?* Then the Jacobian would be close to the identity and the gradient would flow back nearly unchanged. This is precisely the design that LSTMs adopt for the cell state, and later the design that residual connections adopt in Transformers. The same trick appears twice in this syllabus — notice it.

1.5 Bidirectional RNNs

What it is. Two independent RNNs over the same sequence, one running left to right and one right to left, whose hidden states are concatenated at each position.

The idea, plainly. To tag the word “bank” you would like to see both “river” before it and “flooded” after it. One RNN reads the sentence forwards, another reads it backwards, and each position ends up with a representation of its full context: $h^{(t)} = [\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$.

Problem it solves. The “can only see the past” limitation of the simple RNN.

Advantages. Substantially better on tagging tasks (POS, NER, chunking) because disambiguation usually needs right context. A bidirectional GRU is a strong default starting point for sequence tagging.

Disadvantages. Roughly doubles compute and parameters. Crucially, it needs the *entire* sequence before it can produce any output.

What it cannot solve. It does not help with generation or with streaming. It does not fix vanishing gradients — each direction still suffers from them.

Use it when. Offline tasks where the full input is available: classification, tagging, encoding the source side of a translation system.

Do not use it when. Real-time/streaming decoding, or any autoregressive generation task — the backward pass would let the model see the future tokens it is supposed to predict, which is leakage.

Counterfactual. *What if we made a language model bidirectional?* It would be trivially able to cheat: predicting token t while its backward pass has already read token t . This is exactly why BERT cannot simply be trained left-to-right bidirectionally and instead has to invent *masking* (Section 4.2). The bidirectional/generation tension is a recurring exam theme.

One loss or two? One combined loss. The two directions each produce a hidden state at every position, and both feed a single shared output layer:

$$o^{(t)} = \text{softmax}\left(V_f h_f^{(t)} + V_b h_b^{(t)}\right).$$

One prediction per position gives one loss per position gives one total loss. Where the two RNNs *do* separate is backpropagation: gradients from that shared loss flow backwards independently through the forward RNN (updating W_f, U_f) and the backward RNN (updating W_b, U_b). The two networks never talk to each other during the forward pass either — they run independently and meet only at the combination step.

Why “bidirectional” necessarily means two networks. There is no single RNN that can run in both directions at once; the “bi” literally means two networks with their own weight matrices. This matters architecturally: the moment you want a bidirectional encoder you have already committed to multiple components, and since a decoder cannot be bidirectional, the encoder–decoder split is not merely an engineering convenience — it is the structural consequence of wanting bidirectional reading alongside unidirectional writing.

Worked comparison on the same sentence. Running the earlier tagging example bidirectionally (same weights, $h_b^{(6)} = \mathbf{0}$ for the backward start) gives at position 1:

$$\begin{aligned} V_f h_f^{(1)} + V_b h_b^{(1)} &= [0.3808, 0, 0, 0, 0] + [0.3808, 0.1817, 0.0899, 0.0448, 0.0224] \\ o^{(1)} &= [0.3293, 0.1844, 0.1682, 0.1608, 0.1572], \quad L^{(1)} = 1.1107 \end{aligned}$$

against 1.3173 for the unidirectional model. Across the sentence, $L_{\text{total}} = 1.3602$ versus 1.4394. Lower, because every word now sees both sides. Note also the asymmetry at the edges: $h_b^{(1)}$ (“the”) has read the whole sentence backwards, whereas $h_b^{(5)}$ (“great”) has seen only itself — it was the backward RNN’s first step.

2 Part II: Encoder–Decoder and Attention

2.1 The Sequence-to-Sequence (Encoder–Decoder) Model

What it is. Two recurrent networks joined together. An **encoder** RNN reads the entire input sequence and compresses it into a fixed-size vector called the *context vector* (or thought

vector). A **decoder** RNN is initialised with that vector and generates the output sequence one token at a time.

The idea, plainly. Translation is not a tagging problem: “I am hungry” (3 words) becomes “J’ai faim” (2 words), and the word order need not match. So you cannot emit one output per input. Instead: read the whole English sentence, form a single mental summary, then write the French sentence from that summary. The encoder is the reader, the context vector is the summary, the decoder is the writer.

Mechanics.

1. Encoder consumes $x_1 \dots x_n$; keep the final hidden state $c = h_{\text{enc}}^{(n)}$.
2. Initialise decoder state with c ; feed a start token `< sos >`.
3. At each decoder step, produce a distribution over the vocabulary, emit a token, and feed that token back in as the next input.
4. Stop at `< eos >`.

During training the decoder is fed the *ground-truth* previous token rather than its own prediction (**teacher forcing**); at inference it must consume its own outputs (**autoregressive generation**). The mismatch between these two regimes is called *exposure bias*.

Problem it solves. Sequence transduction with unequal, unaligned input and output lengths: machine translation, summarisation, dialogue, speech recognition, question answering.

Advantages. Fully general over length pairs; end-to-end trainable; no need for hand-built alignment or phrase tables as in statistical MT.

Disadvantages.

- **The information bottleneck.** The whole source sentence, however long, must fit into one fixed-size vector. Quality degrades sharply as sentences get longer.
- The encoder’s final state is biased toward the end of the source sentence.
- Still sequential on both sides; still suffers exposure bias.
- Errors compound during generation: one wrong token corrupts everything after it.

What it cannot solve. Long inputs. A 50-word legal sentence and a 5-word greeting get the same number of bits of context.

What it gave rise to. Attention, invented precisely to remove this bottleneck.

Use it when. Any input-sequence to output-sequence mapping where the two are not aligned position-by-position.

Do not use it when. The task is aligned tagging (use a bidirectional encoder with per-token outputs; a decoder is wasted machinery) or classification (use the encoder alone).

Counterfactual. *What if the context vector were passed to every decoder step instead of only initialising the first one?* This is in fact a common variant and it helps a little — the decoder stops “forgetting” the source as it generates. But it does not fix the underlying problem, because the vector is still a single fixed-size summary. The real fix has to let the decoder look back at *all* encoder states, which is attention.

The three components, written out.

- **Encoder:** reads $x_1 \dots x_n$, updating its hidden state each step. Its final state h_n^e summarises the whole input.
- **Context vector:** $c = h_n^e$. The *only* bridge between the two halves.
- **Decoder:** starts from $h_0^d = c$ and generates autoregressively,

$$h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, c), \quad z_t = f(h_t^d), \quad y_t = \text{softmax}(z_t),$$

conditioning each step on the previous output token, the previous decoder state, and the context.

Training minimises cross-entropy averaged over all target tokens, $L = \frac{1}{T} \sum_{i=1}^T L_i$ with $L_i = -\log P(y_i)$. A separator token $\langle s \rangle$ is placed between source and target when the two are packed into one stream; the encoder’s outputs are ignored while it reads the source, and generation begins after $\langle s \rangle$.

What the encoder and decoder actually are, architecturally. Neither is a vanilla RNN in practice.

- The standard **encoder** is a *stacked bidirectional LSTM*. LSTM because a vanilla RNN cannot reliably encode a long sentence; bidirectional because every source token’s representation should see both sides; stacked because lower layers capture surface patterns and higher layers capture abstract meaning. The context vector is taken from the top layer’s final state.
- The standard **decoder** is a *stacked unidirectional LSTM*. It keeps the gating and the stacking but *cannot* be bidirectional: at inference the future tokens do not exist yet. It also receives two inputs a plain encoder never handles — the context c and the previously generated token $\hat{y}_{t-1}$.

Two separate problems, often confused. The bottleneck (compressing everything into one vector) and the vanishing gradient (signal dying over many steps) are different failures. LSTMs largely fix the gradient problem. The bottleneck is a *capacity* problem and survives untouched — which is why attention had to be invented on top.

How does the decoder know how many words to produce? It does not, and it never counts. Every target sentence in training is terminated with an end-of-sequence token $\langle /s \rangle$, which lives in the vocabulary like any other word. So the model does not learn to produce “Soy un estudiante” — it learns to produce “Soy un estudiante $\langle /s \rangle$ ”. At every step the question is always the same (“what is the next token?”), and one of the available answers is always $\langle /s \rangle$. Generation stops the first time $\langle /s \rangle$ wins, so a 5-word source can produce a 3-word target with exactly one $\langle /s \rangle$. Source and target lengths are entirely decoupled.

But an LSTM must know how many times to unroll. Correct, and the resolution is the training/inference split:

- **Training:** the gold target is known, so the decoder unrolls exactly as many steps as the target has tokens (including $\langle \text{s} \rangle$). A static unroll.
- **Inference:** no gold target exists, so the decoder runs one step at a time, checks whether the output was $\langle \text{s} \rangle$, and if not feeds it back in and steps again. A dynamic loop.

This is the same distinction as teacher forcing: having the full target in advance is exactly what permits the fixed unroll.

2.2 Attention

What it is. A mechanism that lets the decoder, at each output step, look back over *all* encoder hidden states and build a fresh, weighted summary tailored to the word it is about to generate.

The idea, plainly. A human translator does not memorise the source sentence and then recite a translation. They glance back at the relevant source word for each word they write. Attention gives the model the same ability: while producing the French word for “hungry” it puts most of its weight on the English token “hungry” and little elsewhere.

Mechanics. Keep every encoder state $h_1 \dots h_n$ (do not throw them away). At decoder step t with decoder state s_t :

$$\begin{aligned}
 e_{t,i} &= \text{score}(s_t, h_i) && \text{relevance of source position } i \\
 \alpha_{t,i} &= \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})} && \text{softmax: weights sum to 1} \\
 c_t &= \sum_{i=1}^n \alpha_{t,i} h_i && \text{context vector for this step}
 \end{aligned}$$

The score function may be a dot product ($s_t^\top h$), a scaled dot product, or a small feed-forward network (additive/Bahdanau attention). The context c_t is then concatenated with s_t to predict the output token.

Problem it solves. The fixed-size bottleneck. The context is now *dynamic*: a different summary for every output step, and its size effectively grows with the input because all n states remain available.

Advantages.

- Large quality gains, especially on long sentences.
- Creates a shortcut for gradients: the loss at output step t connects directly to encoder state h_i without travelling through n recurrent steps. This substantially eases the vanishing gradient problem.
- The weights $\alpha_{t,i}$ form a soft alignment matrix, which is interpretable (a partial answer to the black-box complaint).

Disadvantages. Computes n scores per output step, so cost is $O(n \cdot m)$ for input length n and output length m ; memory grows because all encoder states are retained. Interpretability of attention weights is weaker than it looks — attention is correlation, not explanation.

What it cannot solve. Attention on top of an RNN still leaves the RNN’s sequential computation intact. Training is still slow, because h_i still has to be computed one step at a time. This residual complaint is exactly what the Transformer answers.

Use it when. Any encoder–decoder task; effectively always.

Counterfactual. *What if we replaced the softmax with a hard argmax (“hard attention”)?* The model would attend to exactly one source position. This is cheaper at inference and sometimes more interpretable, but the operation is not differentiable, so you would need reinforcement learning or a relaxation to train it. Soft attention is used almost universally because it is differentiable end-to-end — a good illustration of a design chosen for trainability rather than realism.

The three scoring functions, and why you would pick each.

Score	Formula	Note
Dot product	$h_{i-1}^d \cdot h_j^e$	No parameters. Requires encoder and decoder hidden states to be the <i>same size</i> .
Bilinear (Luong)	$h_{i-1}^d W_s h_j^e$	W_s is learned end-to-end. It learns <i>which aspects</i> of similarity matter for the task, and it allows different encoder and decoder dimensions.
Additive (Bahdanau)	a small feed-forward net over the pair	Most expressive, most expensive.

2.3 Worked Example: Computing One Attention Context Vector

Translating “The car is very cool” into Spanish. Five encoder states, 2-dimensional to keep the arithmetic visible. The decoder has just finished step 1 with $h_1^d = [0.5, 0.3]$ and is about to take step 2.

Word	Encoder state
h_1^e “The”	$[0.2, 0.1]$
h_2^e “car”	$[0.9, 0.8]$
h_3^e “is”	$[0.3, 0.4]$
h_4^e “very”	$[0.1, 0.2]$
h_5^e “cool”	$[0.5, 0.6]$

Step 1 — score every encoder state against h_1^d (dot product).

0.13, 0.69, 0.27, 0.11, 0.43

(For instance $0.5 \times 0.9 + 0.3 \times 0.8 = 0.69$ for “car”.)

Step 2 — softmax to get the weights α_{2j} . Exponentiating gives 1.139, 1.994, 1.310, 1.116 and 1.537, which sum to 7.096, so

$\alpha_2 = [0.16, 0.28, 0.18, 0.16, 0.22]$ for (The, car, is, very, cool),

which sums to 1. “car” wins because its encoder state pointed most nearly in the same direction as the decoder’s state.

Step 3 — weighted sum.

$$c_2 = 0.16[0.2, 0.1] + 0.28[0.9, 0.8] + 0.18[0.3, 0.4] + 0.16[0.1, 0.2] + 0.22[0.5, 0.6] = [0.47, 0.48].$$

This c_2 now enters the decoder alongside h_1^d and $\hat{y}_1$ to produce h_2^d . The whole process restarts at step 1 for the next decoder step, using the newly produced h^d .

Attention is a distribution, not a selection. A common misreading is that the decoder “picks” the highest-weighted source word. It does not — *all five* contributed to c_2 . What changes between steps is where the weight is concentrated. When translating “very cool” into “muy genial”, the weights at that step might be $[0.05, 0.08, 0.07, 0.38, 0.42]$: spread across two words, so c blends both. Sharply peaked versus spread out — that is the real distinction.

What is interpretable and what is not. The *weights* α_{ij} are interpretable and are what people visualise as alignment heatmaps. The *context vector* c_i is not: its individual dimensions are learned, distributed, entangled features, and dimension 1 does not mean “subject”.

The encoder computes no loss of its own. Loss is computed only at the decoder — cross-entropy between predicted and gold target words, one per position, averaged into $L = \frac{1}{T} \sum_i L_i$, then a single backward pass. But the encoder’s weights are absolutely updated: gradients flow backwards through the decoder, through the attention mechanism, and on into the encoder. The encoder is trained *indirectly*, and the whole system is one model with one loss.

Another reason attention helps. Without attention the only gradient path into the encoder is through the single context vector. With attention, gradients reach *all* encoder hidden states through the attention weights, so the encoder gets a far richer training signal.

Is it unfair to blame the encoder for the decoder’s mistakes? This is the credit assignment question, and the answer is that the chain rule handles it automatically:

$$\frac{\partial L}{\partial W_e} = \frac{\partial L}{\partial(\text{decoder output})} \cdot \frac{\partial(\text{decoder output})}{\partial c} \cdot \frac{\partial c}{\partial W_e}.$$

If the encoder’s contribution to the error was small, the middle term is small and the gradient reaching the encoder is proportionally small. No special fix is needed. More fundamentally, there is no independent definition of “good encoder output”: the encoder’s job is not to produce some universally correct representation but to produce one that is useful *to this decoder*. The two co-adapt as a single jointly optimised system.

2.4 Why GPUs Matter (and Why Recurrence Became Unacceptable)

The hardware argument. CPUs are latency-optimised: a few very fast cores, designed to finish a single instruction chain quickly. GPUs are bandwidth-optimised: thousands of simple cores plus very fast VRAM, designed to perform the same operation on enormous amounts of data at once.

Why deep networks fit GPUs. A neural layer is a matrix multiplication — many identical neurons performing the same computation on different data. That is precisely the workload a GPU is built for. The more arithmetic per unit of data movement, the larger the GPU’s advantage. Massive thread parallelism also hides memory latency.

Why this doomed the RNN. An RNN’s time axis is a strict dependency chain: you cannot compute $h^{(5)}$ before $h^{(4)}$. So the one axis with the most work in it cannot be parallelised. As GPUs got faster, RNN training time barely improved. A model that can compute all positions simultaneously would convert hardware progress directly into research progress. That model is the Transformer, and the observation above is the real motivation behind “Attention Is All You Need”.

3 Part III: The Transformer

3.1 Overview

What it is. An encoder–decoder architecture that removes recurrence and convolution entirely, and relies only on attention plus position-wise feed-forward layers. Introduced in *Attention Is All You Need* (Vaswani et al., 2017).

The idea, plainly. If attention already lets a decoder look anywhere in the source, why not let *every* token look at every other token directly, and drop the recurrent chain altogether? Then all positions can be computed in one parallel matrix multiplication. The price is that, without recurrence, the model has no idea what order the tokens came in — so order must be injected separately.

The stack, in the original paper.

- Encoder: $N = 6$ identical layers. Each layer = multi-head self-attention $\rightarrow$ Add & Norm $\rightarrow$ position-wise feed-forward network $\rightarrow$ Add & Norm.
- Decoder: $N = 6$ identical layers. Each layer = *masked* multi-head self-attention $\rightarrow$ Add & Norm $\rightarrow$ encoder–decoder (cross) attention $\rightarrow$ Add & Norm $\rightarrow$ feed-forward $\rightarrow$ Add & Norm.
- Dimensions: $d_{\text{model}} = 512$, $h = 8$ heads, $d_k = d_v = 64$, $d_{\text{ff}} = 2048$. Every sub-layer and the embedding layer outputs d_{model} , which is what makes residual connections possible.
- Every sub-layer is wrapped as $\text{LayerNorm}(x + \text{Sublayer}(x))$.

Yes, the encoders and decoders are neural networks. An “encoder” is not some new kind of object — it is one architectural block made of a multi-head self-attention sub-layer, an Add & Norm, a feed-forward network, and another Add & Norm. “Six encoders” means that *same block design* is repeated six times, stacked one on top of the next. Each copy has its **own separate weights**; they are six instances of one design, not one network run six times. And there are no RNNs or LSTMs anywhere inside — removing them is the entire point of the paper.

Why stack six? Each layer builds a more abstract representation of the same tokens: early layers tend to capture syntax, later layers semantics, in the same way a deep CNN goes edges $\rightarrow$ textures $\rightarrow$ objects. Six was an empirical choice. More layers give more capacity, with diminishing returns and rising cost.

Embedding happens only in the bottom encoder. This confuses many readers of Almar’s blog. Every encoder in the stack consumes a list of d_{model} -sized vectors and emits a list of d_{model} -sized vectors. For the *bottom* encoder those input vectors are the word embeddings plus positional encodings. For every encoder above it, the input is simply the output of the

encoder below — already contextualised vectors, not tokens. There is nothing left to look up in an embedding table, so embedding happens once, at the bottom. This uniform “vectors in, vectors out” interface is exactly what makes the blocks stackable.

Is the Transformer a seq2seq model? Yes. It is an encoder–decoder that maps an input sequence to an output sequence; what changed is the internals, not the shape of the task.

3.2 Self-Attention

What it is. Attention where the queries, keys and values all come from the same sequence: each token attends to every token in its own sentence, including itself.

Why “self”. Ordinary attention asks “which source words matter for this target word?” — the query comes from a different sequence and often a different task. Self-attention asks “which words in *this same* sentence matter for understanding *this* word?”. In “The animal didn’t cross the street because **it** was too tired”, self-attention is what lets “it” bind to “animal” rather than “street”. Trivial for a human, historically very hard for a machine.

The Q/K/V metaphor (from information retrieval). Each token is linearly projected three times:

- **Query** — the search string you type into YouTube.
- **Key** — the video titles the system matches your string against.
- **Value** — the actual videos returned to you.

Ranking by query–key similarity and returning a blend of the corresponding values is exactly what we want: score every other word (key) against the current word (query), then mix their contents (values) accordingly.

Mechanics. With $X \in \mathbb{R}^{n \times d_{\text{model}}}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Step by step:

1. $QK^\top$ gives an $n \times n$ matrix of raw similarity scores (dot product = how aligned two vectors are).
2. Divide by $\sqrt{d_k}$ — **scaling**. With $d_k = 64$ this is a division by 8.
3. Softmax each row so the weights over the sentence sum to 1.
4. Multiply by V to get, for each token, a weighted blend of all tokens’ values.

Note that $d_k = 64$ is deliberately smaller than $d_{\text{model}} = 512$: each head works in a compressed subspace, so that $h = 8$ heads together cost about the same as one full-width attention.

Problem it solves. Direct, constant-length paths between any two positions. In an RNN, information from token 1 reaches token 50 through 49 sequential transformations; in self-attention it is one hop. It also makes the whole sequence computable in parallel, because the operation is just three matrix multiplications and a softmax.

Advantages.

- Maximum path length between any two tokens is $O(1)$: excellent for long-range dependencies.
- Fully parallel across positions: converts GPU throughput directly into training speed.
- Content-based, dynamic routing: what attends to what depends on the actual input, not on fixed positions as in a convolution.

Disadvantages.

- $O(n^2)$ time and memory in the sequence length, because of the $n \times n$ score matrix. This is the single biggest cost driver in modern LLMs.
- Permutation invariant on its own — it has no notion of order.
- No built-in locality bias, so it is data-hungry: it must learn from scratch that nearby words are usually more relevant.

Counterfactual 1. *What if we dropped the $\sqrt{d_k}$ scaling?* For large d_k the dot products have large variance and grow in magnitude. Feeding large values into softmax pushes it into a saturated regime — one weight near 1, the rest near 0 — where its gradient is almost zero. Training would stall or become unstable. The scaling keeps the logits in a range where softmax has usable gradients. Expect this as an exam question.

Counterfactual 2. *What if Q , K and V were the same vectors, with no projections?* The score matrix would be $XX^\top$, which is symmetric, so “A attends to B” would always equal “B attends to A”. Real linguistic relations are asymmetric — a pronoun depends on its antecedent, not the reverse. Separate W^Q and W^K break the symmetry. A separate W^V additionally lets the model decide that the information worth *passing on* differs from the information used for *matching*.

3.3 Worked Example: Attention for a Single Word

The intuition first. Read the sentence “The trophy didn’t fit in the suitcase because **it** was too big.” You worked out instantly what “it” refers to, because your brain paid more attention to “trophy” than to “suitcase”. That is all attention does: it lets every word look at every other word and ask, *who here is most relevant to me?*

The numbers. Sentence “The cat sat”; compute the attention output for “cat”. Use 2-D vectors so the arithmetic is doable by hand — real Transformers use 64, but the mathematics is identical. Assume the projections have already been applied:

$$q_2 = [1, 1] \text{ (cat's query); } \quad k_1 = [1, 0], \quad k_2 = [0, 1], \quad k_3 = [1, 1];$$
$$v_1 = [1, 0], \quad v_2 = [0, 1], \quad v_3 = [1, 1].$$

Step 1 — raw scores ($q_2 \cdot k_j$).

$$\text{score}(\text{The}) = 1, \quad \text{score}(\text{cat}) = 1, \quad \text{score}(\text{sat}) = 2.$$

“sat” scores highest, which is linguistically sensible.

Step 2 — scale by $\sqrt{d_k} = \sqrt{2} \approx 1.414$. [0.707, 0.707, 1.414].

Step 3 — softmax. $e^{0.707} \approx 2.028$ (twice) and $e^{1.414} \approx 4.113$, summing to 8.169, giving

$$\alpha = [0.248, 0.248, 0.504]$$

— 24.8% attention on “The”, 24.8% on itself, 50.4% on “sat”.

Step 4 — weighted sum of the values.

$$z_2 = 0.248[1, 0] + 0.248[0, 1] + 0.504[1, 1] = [0.752, 0.752].$$

“cat” is no longer “cat” in isolation: its new vector is a blend, pulled mostly toward “sat”. In the lecture slides this output is written z for one word and Z for the whole sentence at once, with $Z = AV$.

Mapping each step back to the formula.

Formula piece	What we did
$QK^\top$	dot products $\rightarrow [1, 1, 2]$
$\div \sqrt{d_k}$	divide by 1.414 $\rightarrow [0.707, 0.707, 1.414]$
$\text{softmax}(\cdot)$	$\rightarrow [0.248, 0.248, 0.504]$
$\times V$	weighted sum $\rightarrow [0.752, 0.752]$

The scaling argument, stated precisely. $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k terms. If the components have mean 0 and variance 1, the dot product has mean 0 but **variance** d_k — it grows linearly with the dimension. Dividing by $\sqrt{d_k}$ scales the variance back to $\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \propto \frac{d_k}{d_k} = 1$, and it is the unique scaling with that property. Without it, at $d_k = 512$ the logits are large, the softmax saturates into a near one-hot distribution, and its gradient goes to zero.

A counting clarification. For the sentence “There is a cat” with one head, each *word* gets one q , one k and one v — 12 vectors. But those are stacked into **3 matrices** per head (Q, K, V), each $4 \times d_k$, one row per word. Each head has its own three *weight* matrices W^Q, W^K, W^V ; Q, K, V are the results of applying them, not things a word owns individually.

3.4 Multi-Head Attention

What it is. Run h self-attention operations in parallel with different learned projections, concatenate their outputs, and pass the result through a final linear layer W^O .

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O,$$

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V).$$

The idea, plainly. A single attention distribution has to commit to one weighting. But “it” may need to attend to its antecedent *and* the current verb *and* the sentence topic at the same time. One head cannot express three different weightings simultaneously; averaging them muddles all three. Multiple heads let the model maintain several relationship types in parallel — one head may track syntactic subjects, another coreference, another local adjacency. As the lecture puts it: multiple heads can (hopefully) focus on more things than one head can.

Advantages. Richer representations at essentially no extra cost, since each head is narrower ($d_k = d_{\text{model}}/h$). Also gives redundancy: heads can be pruned later.

Disadvantages. Many heads are demonstrably redundant and can be removed with little loss — so the mechanism is not as efficient as it appears. Head interpretations are suggestive, not reliable.

Counterfactual. *What if we used one head of full width d_{model} instead of eight of width 64?* Parameter count would be similar, but the layer could represent only one attention pattern per layer. Empirically this is clearly worse. The lesson is that the benefit comes from having *multiple simultaneous* attention distributions, not from extra parameters.

The heads run in parallel, not one after another. A common misstatement is “compute z once, then seven more times”. All h heads run simultaneously, each with its own W^Q, W^K, W^V , producing $Z_1 \dots Z_h$ at the same time. The feed-forward network expects a single matrix, so the heads’ outputs are concatenated side by side and multiplied by W^O to compress back to d_{model} — only then does the result go to the FFN.

3.5 Worked Example: Multi-Head Dimensions

Setup. Sentence “The cat sat” (3 words), $d_{\text{model}} = 4$, 2 heads, so $d_k = d_{\text{model}}/h = 4/2 = 2$. (Real Transformer: 512, 8, 64.) The input X is 3×4 .

Per head.

$$Q = XW^Q : (3 \times 4)(4 \times 2) = (3 \times 2), \quad \text{same for } K, V.$$

$$QK^\top : (3 \times 2)(2 \times 3) = (3 \times 3) \text{ scores} \rightarrow \text{scale} \rightarrow \text{softmax} \rightarrow Z = AV : (3 \times 3)(3 \times 2) = (3 \times 2).$$

Combining. Concatenate $Z_1 \| Z_2$ along the columns: (3×4) — back to the original width. Then multiply by W^O of shape (4×4) , giving Z_{final} of shape (3×4) : *exactly the shape of the input X* . That is what allows blocks to stack and residual connections to work.

Thing	Shape	Why
X	$n \times d_{\text{model}}$	n words, each d_{model} wide
$W^Q/W^K/W^V$ per head	$d_{\text{model}} \times d_k$	projects down to this head’s slice
Q, K, V per head	$n \times d_k$	one row per word
Z per head	$n \times d_k$	same width as V
Concatenated	$n \times (h \cdot d_k) = n \times d_{\text{model}}$	the key identity
After W^O	$n \times d_{\text{model}}$	back to the original width

The relationship $h \cdot d_k = d_{\text{model}}$ is what makes the dimensions line up end-to-end. With 8 heads in the real model: $8 \times 64 = 512$.

Where the numbers in W^Q ’s shape come from. If W^Q is (4×2) , the **4 rows** exist because each word’s input vector has 4 numbers — the matrix must accept d_{model} numbers in. The **2 columns** exist because $d_k = 2$, not directly “because there are 2 heads”. The head count influences it only indirectly, through $d_k = d_{\text{model}}/h$. With 4 heads, d_k would be 1 and W^Q would be (4×1) .

A tiny end-to-end run. With $d_{\text{model}} = 2$, 2 heads, $d_k = 1$, and $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, let head 1 project onto the first coordinate and head 2 onto the second. Head 1’s scores make “The” and “sat” attend to each other (both have a 1 in slot 1); head 2’s make “cat” and “sat” attend to each other. The outputs are

$$Z_1 = [0.84, 0.66, 0.84]^\top, \quad Z_2 = [0.66, 0.84, 0.84]^\top,$$

concatenating to $\begin{bmatrix} 0.84 & 0.66 \\ 0.66 & 0.84 \\ 0.84 & 0.84 \end{bmatrix}$. Two heads, two different relationships discovered at the same time — which is the entire argument for multi-head attention, made concrete.

3.6 Positional Encoding

What it is. A vector added to each input embedding that encodes the token’s position in the sequence.

Why it is necessary. Self-attention treats its input as a *set*. Shuffle the tokens and every attention score is unchanged; the outputs just permute along with the inputs. Without positional information the Transformer is a sophisticated bag of words, and “dog bites man” equals “man bites dog”.

Mechanics (sinusoidal). For position k , embedding dimension d , index $0 \leq i < d/2$, and a scalar n set to 10000 in the paper:

$$PE(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \quad PE(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right).$$

Properties worth memorising:

- Same dimension as the embedding, so it can simply be added.
- Each dimension is a sinusoid of a different wavelength, from short (fast-varying, fine position) to very long (slow-varying, coarse position) — like the hands of a clock at different speeds.
- Because it is a fixed periodic function, the encoding of position 7 is the same in every example regardless of sentence length, and the model can extrapolate somewhat to lengths not seen in training.
- $PE_{k+\delta}$ is a fixed linear function of PE_k , so relative offsets are easy for the model to compute.
- Positional encodings can also be *learned*; in practice learned embeddings did not perform better in the original experiments, and they cannot extrapolate beyond the trained maximum length.

Counterfactual 1. *What if we omitted positional encoding entirely?* The model becomes permutation-invariant. It could still do bag-of-words tasks like topic classification reasonably, but syntax, negation scope, and translation would collapse.

Counterfactual 2. *What if we used a simple integer position, $p_k = k$?* The magnitude would grow without bound, dominating the embedding and destabilising training, and the network would have to learn to decode a scalar into useful relative-distance features. The sinusoidal scheme is bounded, smooth, and already exposes multiple scales.

Counterfactual 3. *What if position were concatenated instead of added?* This works and is arguably cleaner conceptually, but it costs extra dimensions in every subsequent layer. Addition is chosen for efficiency; the model can learn to allocate subspaces to content versus position.

Where it sits in the pipeline. Token $\rightarrow$ embedding $\rightarrow$ *add* positional encoding $\rightarrow$ attention. It happens **once, at the input**, not before every attention layer. (Contrast RoPE later, which is applied inside every layer.)

Why sine *and* cosine, paired? Not because even and odd indices are special — the even/odd split is just the paper’s implementation convention. The reason is the *pairing*. If you used sine alone you would lose uniqueness, because $\sin(\theta) = \sin(\pi - \theta)$, so two different positions could produce the same value. A $(\sin, \cos)$ pair is like a point (x, y) on a unit circle: it pins down the angle uniquely. Each pair at frequency i is a clock hand rotating at its own speed; stack many clocks at different speeds and every position gets a unique fingerprint.

The deeper reason: relative positions become a fixed linear operation. From the angle-addition identities,

$$\begin{aligned}\sin((p+k)\omega) &= \sin(p\omega)\cos(k\omega) + \cos(p\omega)\sin(k\omega), \\ \cos((p+k)\omega) &= \cos(p\omega)\cos(k\omega) - \sin(p\omega)\sin(k\omega),\end{aligned}$$

which written as a matrix product is exactly

$$PE(p+k) = M_k \cdot PE(p), \quad M_k = \begin{pmatrix} \cos k\omega & \sin k\omega \\ -\sin k\omega & \cos k\omega \end{pmatrix}, \quad PE(p) = \begin{pmatrix} \sin(p\omega) \\ \cos(p\omega) \end{pmatrix}.$$

Every entry of M_k depends only on k and ω — **not on p** . So the same matrix takes you two steps ahead from *any* starting position. M_k is a rotation matrix, which is the clock-hand picture stated algebraically, and it comes from trigonometry, not from training.

What that buys you, concretely. Take “The cat sat on the mat”. With $k=2$, M_2 maps $PE(1)$ (“cat”) to $PE(3)$ (“on”), and the very same M_2 maps $PE(2)$ (“sat”) to $PE(4)$ (“the”). Without this property, the relationship between positions 1 and 3 would look numerically unrelated to the relationship between positions 50 and 52, even though the gap is identical, and the model would have to learn every pair separately. With it, “two apart” is structurally the same everywhere, so attention can generalise about relative distance instead of memorising position pairs.

A correction worth internalising. It is tempting to say “sin/cos saves us from recomputing $PE(p+k)$ ”. That is not the benefit — positional encodings are precomputed for all positions before training even starts, so there is no runtime recomputation to avoid. The real benefit is about the *attention mechanism*: relative distances are structurally consistent, so relative position is something the model can generalise rather than memorise.

3.7 Residual Connections and Layer Normalisation (Add & Norm)

What it is. Around every sub-layer, the input is added to the output and the sum is normalised: $\text{LayerNorm}(x + \text{Sublayer}(x))$.

The “Add” part. The residual connection provides an identity path from input to output. Two benefits:

- **Preserves information.** Attention re-mixes tokens; without the shortcut, the original token identity would be progressively washed out across 6 (or 96) layers.
- **Handles vanishing gradients.** The derivative through $x + f(x)$ is $1 + f'(x)$, so gradient can flow straight back through the 1. This is the same additive highway idea as the LSTM cell state — and it is why 100-layer Transformers are trainable at all.

Concretely, in the first encoder layer the “Add” literally adds the multi-head attention output to the (embedding + positional encoding) input.

The “Norm” part: layer normalisation. Normalise each vector to zero mean and unit variance *across its own feature dimensions*, then rescale with learned gain and bias. Contrast with batch normalisation, which computes mean and variance for each neuron across a mini-batch of training cases. Batch norm dramatically reduces training time in feed-forward networks, but it is a poor fit here:

- Sequences have variable length and heavy padding, so per-neuron batch statistics are noisy and ill-defined.
- Batch norm behaves differently at training and inference time, and depends on batch size — awkward for autoregressive decoding one token at a time.
- Layer norm is computed per token, independently of the batch, so it behaves identically in training and inference.

Pre-norm vs post-norm. The original paper normalises *after* the residual addition (post-norm). Modern large models (including LLaMA) normalise the *input* of each sub-layer instead (pre-norm), because post-norm requires careful learning-rate warm-up and becomes unstable at great depth. Pre-norm keeps a completely clean residual path from the first layer to the last.

Counterfactual. *What if we removed the residual connections?* Deep stacks would become untrainable: gradients would vanish and the token’s own identity would be lost after a few rounds of attention mixing. Empirically, removing residuals from a Transformer is catastrophic, far more so than removing multi-head structure.

“But the original X has no attention in it — doesn’t adding it back spoil the attention output?” No, and the reason is that the attention sub-layer is trained *knowing* the addition will happen. $\text{MHA}(X)$ is not trying to be a complete new representation; it is optimised to produce *what needs to be added* to X to improve it. It is a correction, a delta, not a rewrite. X carries token identity and position; $\text{MHA}(X)$ carries contextual updates; $X' = X + \text{MHA}(X)$ carries both, and neither cancels the other. If you discarded X you would actually risk *losing* precise token identity, because attention is a weighted average over all positions and averaging blurs. Where X already captures something well, the model learns to add only a small correction; where context matters more, attention contributes more. The balance is learned.

“Doesn’t normalisation destroy information?” Suppose $[2, 4, 5, 1]$ becomes something like $[0.3, 0.45, 0.7, 0.1]$. What is *preserved* is the relative structure — which dimensions are large, which are small, and roughly by how much. The fact that the third element dominated and the fourth was weakest survives intact. What is *lost* is the absolute magnitude: afterwards you cannot tell whether the original values were around 5 or around 500.

Why losing absolute magnitude is safe. Meaning in vector representations lives in *direction*, not length — similarity between word vectors is measured by cosine similarity, which depends only on the angle. So two vectors pointing the same way mean the same thing whether their magnitudes are 1 or 1000. (One precision: layer norm also mean-centres, so it is not strictly angle-preserving in the way pure L2 normalisation would be. The practical effect is the same.)

And anything genuinely needed can be restored. After normalising, layer norm applies two *learned* parameters per dimension: a scale γ and a shift β . If some particular scale really did matter, the model learns to put it back through γ . Normalisation puts everything on a clean, stable baseline; γ and β then reshape it into whatever the network actually wants.

3.8 The Position-wise Feed-Forward Network

What it is. A two-layer MLP applied independently and identically to every position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2,$$

with inner dimension $d_{\text{ff}} = 2048$ against $d_{\text{model}} = 512$.

Why it is there. Attention is essentially a weighted *average* — a linear operation once the weights are fixed. Stacking averages without nonlinearity would collapse. The FFN provides the per-token nonlinear processing: attention gathers information from other tokens, the FFN then thinks about it. A useful slogan: *attention mixes across tokens, the FFN mixes across features*. It also holds most of the parameters (roughly two-thirds of a Transformer block), and is widely believed to store much of the model’s factual knowledge — which is why Mixture-of-Experts models make *this* layer the expert layer.

What “position-wise” actually means. The *same* MLP — the same W_1, b_1, W_2, b_2 — is applied to every token’s vector, but each token is processed **independently**, with no information flowing between positions. While the FFN is processing token x_3 , it has no idea what x_1 or x_2 look like. Weights shared across positions; computation separate per position.

Why that is useful even though tokens do not interact. The interaction already happened. By the time a vector reaches the FFN, attention has already mixed in context from other tokens. The FFN’s job is to *transform that individual representation* — refine it with a learned non-linear function — not to mix across positions again. Attention is the **communication** step; the FFN is the **computation** step.

Parameter accounting. Ignoring biases, the two matrices are $d_{\text{model}} \times 4d_{\text{model}}$ and $4d_{\text{model}} \times d_{\text{model}}$, so $8d_{\text{model}}^2$ parameters per block. For $d_{\text{model}} = 768$ that is about 4.72M per block, or roughly 56.6M across 12 blocks — the majority of a GPT-1-sized model’s 120M, before attention and embeddings are even counted.

3.9 The Decoder

Three distinguishing features.

1. **Right-shifted input.** The target sequence is offset by one position, so that when predicting token i the decoder has been given tokens $1 \dots i-1$. This is what makes autoregression well-defined.

2. **Masked multi-head self-attention.** Positions are forbidden from attending to *subsequent* positions. Implemented by setting the corresponding scores to $-\infty$ before the softmax, so their weights become exactly zero. Combined with the right shift, this guarantees that the prediction for position i depends only on outputs before i .
3. **Encoder–decoder (cross) attention.** An extra sub-layer where the *queries* come from the decoder and the *keys and values* come from the encoder output. This is the classical seq2seq attention, now expressed in Q/K/V form: use the current partial translation as query to look up relevant source content.

Final linear and softmax layer. The decoder’s top-layer vector is projected by a linear layer to vocabulary size (the “logits” vector), then softmax converts it into a probability distribution. Greedy decoding takes the argmax; beam search keeps several hypotheses; sampling with temperature and top- k /top- p is used for creative generation.

Counterfactual. *What if we removed the causal mask during training?* The model would see the answer while predicting it, achieve near-zero training loss, and be completely useless at generation time, when future tokens do not exist. The mask is not a performance trick — it is what makes the training objective match the inference procedure.

“Outputs (shifted right)”, made concrete. Suppose the correct translation is “I am happy”. During training the target is offset by one position before being fed in:

Decoder input (shifted right)	Should predict
[START]	I
[START] I	am
[START] I am	happy
[START] I am happy	[END]

Without the shift, position t would have the answer sitting in its own input and could simply copy it. The shift enforces: *predict the next token from everything before it*.

At inference the shift disappears, but the structure survives. There is no target to shift. The decoder starts from [START], predicts a token, appends it, and steps again — which produces exactly the same input/output pairing as the table above, one step at a time instead of all at once. “Shifted right” is a training-time description; the autoregressive loop replaces it at test time.

So during training the whole sentence really is generated at once. Yes. All positions are predicted in one forward pass. The mask is what makes that safe: at position 2, even though “am” and “happy” are physically present in the input matrix, the mask blocks attention to them, so position 2 effectively sees only [START] I. Inference is inherently sequential because you do not have the target; training does not have to be, because you do. That is the whole efficiency argument for teacher forcing plus masking — one forward pass computes the loss for every position, instead of running the decoder T times.

3.10 Worked Example: Masked Self-Attention in the Decoder

Setup. Target tokens <SOS>, “El”, “perro”; $d_{\text{model}} = 4$, 2 heads, so $d_k = 2$. For head 1, suppose the projections give $Q_1 = K_1 = V_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$.

Raw scores, then scaling by $1/\sqrt{2} \approx 0.707$.

	k_0	k_1	k_2
q_0	0.71	0.00	0.71
q_1	0.00	0.71	0.71
q_2	0.71	0.71	1.41

Where exactly the masking happens: after scaling, before softmax. You add a mask matrix whose future entries are $-\infty$ and whose current/past entries are 0:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}, \quad \text{so} \quad \text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

Because $e^{-\infty} = 0$, those positions contribute *exactly* zero — not merely a small weight, but none at all.

Softmax, row by row.

	<SOS>	El	perro
<SOS>	1.000	0.000	0.000
El	0.330	0.670	0.000
perro	0.249	0.249	0.502

Row 0 has only one finite value, so it is forced to 1.0 — the first token can only attend to itself. Then $Z_1 = A_1 V_1$ gives $[1.000, 0.000]$, $[0.330, 0.670]$ and $[0.751, 0.751]$ for the three positions.

What the mask enforced.

Token	Could attend to	Could not attend to
<SOS>	only itself	El, perro
El	<SOS>, El	perro
perro	<SOS>, El, perro	—

The concatenated output then goes to Add & Norm, and from there into cross-attention, where it will query the encoder’s output.

3.11 How the Encoder and Decoder Stacks Are Wired Together

Why e_6 ’s output goes to *every* decoder layer. Each decoder block has a cross-attention sub-layer, which needs K and V from the encoder and Q from itself. The encoder’s work is completely finished after e_6 — that is its richest understanding of the source. So e_6 ’s output is turned into K and V once, and those same K and V are made available to d_1 through d_6 . Every decoder layer independently needs to ask “which part of the source matters right now?”.

Why $d_2 \dots d_6$ also take input from the layer below. That is just stacking, exactly as on the encoder side. d_1 's output is d_2 's input, and so on, each block refining the representation a little further. So d_n takes the output of d_{n-1} , runs it through masked self-attention, then cross-attention (using e_6 's K and V), then the FFN, and hands the result up to d_{n+1} .

d_1 does not produce words. No individual decoder block does. Each block outputs a *matrix of hidden representations* of the same shape as its input — one vector per target position — now enriched by masked self-attention, cross-attention and the FFN. Token prediction happens only once, after d_6 , through the linear and softmax layers. Think of it as layers of abstraction: d_1 might learn rough source–target alignments and basic syntax, the middle layers refine those into nuanced relationships, and d_5/d_6 produce highly abstract representations ready for prediction. Exactly the same logic as the encoder needing six layers.

3.12 Worked Example: The Final Linear and Softmax Layers

Setup. d_6 's output is one 4-dimensional vector per position: $H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$ for $\langle \text{SOS} \rangle$, “El”, “perro”. Vocabulary of 6: $\{\langle \text{SOS} \rangle, \langle \text{EOS} \rangle, \text{El}, \text{perro}, \text{se}, \text{sentó}\}$.

Linear. A learned $W \in \mathbb{R}^{4 \times 6}$ turns each d_{model} -vector into one score (logit) per vocabulary word. Suppose it yields

	$\langle \text{SOS} \rangle$	$\langle \text{EOS} \rangle$	El	perro	se	sentó
$\langle \text{SOS} \rangle$	0.1	0.1	2.0	0.1	0.1	0.1
El	0.1	0.1	0.1	2.0	0.1	0.1
perro	0.1	0.1	0.1	0.1	2.0	0.1

Softmax, row by row and independently. For the first row, $e^{0.1} \approx 1.105$ and $e^{2.0} \approx 7.389$, so the sum is $5(1.105) + 7.389 = 12.914$ and $P(\text{El} \mid \langle \text{SOS} \rangle) = 7.389/12.914 \approx 0.572$, with everything else at ≈ 0.086 .

Each row predicts the *next* token. This is the critical reading: $\langle \text{SOS} \rangle \rightarrow \text{El}$, $\text{El} \rightarrow \text{perro}$, $\text{perro} \rightarrow \text{se}$. All three happen simultaneously during training.

Loss. Cross-entropy at each position against the true next token, then averaged:

$$\mathcal{L} = -\frac{1}{3} [\log(0.572) + \log(0.572) + \log(0.572)] \approx 0.557.$$

Where the loss is and is not computed. It is computed **once**, after the final decoder layer passes through linear + softmax. Layers $d_1 \dots d_5$ only transform representations; they produce no vocabulary distribution, so there is nothing to score them against. But every layer still *receives* gradients. The full backward path is

$$\mathcal{L} \rightarrow \text{Linear} \rightarrow d_6 \rightarrow \dots \rightarrow d_1 \rightarrow \text{cross-attention} \rightarrow e_6 \rightarrow \dots \rightarrow e_1,$$

so the encoder is updated too — its output was used as K and V in every decoder layer's cross-attention, so it participated in producing the loss.

3.13 Which Weights Are Shared, and Which Are Not

This is a favourite question and the answer is surprisingly lopsided: almost nothing is shared.

The one genuine case of sharing: weight tying. A single matrix $W_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ does three jobs:

1. embeds encoder input tokens;
2. embeds decoder input tokens;
3. acts (transposed) as the final pre-softmax linear projection.

Embedding is $W_E \cdot \text{one-hot}$ (pick a row); the output projection is $W_E^\top h$ (project back to vocabulary size). Same weights, opposite direction.

Why that is possible: a shared vocabulary. The original paper trained BPE on the *combined* text of both languages, producing one unified list of roughly 37K subword tokens containing English pieces, French pieces, and pieces common to both (**tion**, say). Token ID 47 therefore means the same subword unit on both sides. If instead you built two separate vocabularies, ID 47 would mean unrelated things in each, and you could not share the embedding matrix between encoder and decoder — though decoder-input-to-output tying would still work.

Everything else is separate.

Component	Relationship
e_1 vs e_2	Completely separate: self-attention (all heads), FFN, LayerNorm
d_1 vs d_2	Completely separate: masked self-attention, cross-attention, FFN, LayerNorm
Head 1 vs head 2 vs head 3 within one module	Each head has its own W_Q^h, W_K^h, W_V^h
Masked self-attention vs cross-attention in the same decoder layer	Two fully independent attention modules, each with its own heads and its own W^O

The paper states it directly: “the linear transformations use different parameters from layer to layer.”

Counting, for a 3-head module. Nine projection matrices ($3 \text{ heads} \times W_Q, W_K, W_V$) plus **one** W^O . Note that W^O is not “shared among the heads” in any meaningful sense — no head ever touches it. The heads finish, their outputs are concatenated, and W^O is applied *once* to the concatenated result.

W^O is not the output linear layer. Different shapes, different places, different purposes. $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ lives *inside* every attention module and resizes concatenated head outputs. $W_E^\top \in \mathbb{R}^{d_{\text{model}} \times |V|}$ sits at the very end of the whole decoder stack and projects to vocabulary logits. They are never shared.

Add & Norm parameters. The “Add” has no learnable weights at all — it is a plain residual addition. The “Norm” has two, γ and β , each of size d_{model} , and *every* LayerNorm instance has its own. So an encoder layer has 2 sets, a decoder layer has 3 sets (one after each of masked self-attention, cross-attention and the FFN). A 2-encoder, 2-decoder model therefore has 4 sets on the encoder side and 6 on the decoder side.

A distinction worth stating precisely: shared weights versus a shared tensor. The output of e_6 is fed into cross-attention in every decoder layer. That is a shared *activation tensor*, not shared weights — each decoder layer projects that same tensor with its own W_K^{cross} and W_V^{cross} . Same input, different treatment.

Full inventory for a 1-encoder, 1-decoder, 1-head model.

- **Shared:** W_E .
- **Encoder:** self-attention W_Q, W_K, W_V, W^O ; LayerNorm γ_1, β_1 ; FFN W_1, b_1, W_2, b_2 ; LayerNorm γ_2, β_2 .
- **Decoder:** masked self-attention W_Q, W_K, W_V, W^O ; LayerNorm γ_1, β_1 ; cross-attention W_Q, W_K, W_V, W^O ; LayerNorm γ_2, β_2 ; FFN W_1, b_1, W_2, b_2 ; LayerNorm γ_3, β_3 .

Positional encodings are fixed sinusoids in the original paper, so they are not weights.

3.14 Byte-Pair Encoding (BPE) and Subword Tokenization

The problem. Word-level tokenization needs a fixed vocabulary. Real language defeats it:

- German compounds: *Krankenhaus* = *kranken* (sick) + *haus* (house) = hospital. Every compound would be a new word.
- English inflection: *low*, *lower*, *lowest* become three unrelated entries.
- Any unseen word maps to <UNK>, destroying information — fatal for names, typos, code and morphologically rich languages.

Character-level tokenization avoids <UNK> but makes sequences very long and forces the model to relearn spelling. Stemming and lemmatisation are not used, because they are language-specific, lossy, and rule-based.

What BPE is. A data-driven compression algorithm turned into a tokenizer. Start from characters (or bytes) and repeatedly merge the most frequent adjacent pair into a new symbol, for a fixed number of merges. Frequent words end up as single tokens; rare words decompose into meaningful pieces.

Worked example (from the lecture).

$$\text{aaabdaaabac} \rightarrow \text{ZabdZabac} \quad (Z=aa) \rightarrow \text{ZYdZYac} \quad (Y=ab) \rightarrow \text{XdXac} \quad (X=ZY)$$

Advantages. Fixed, tunable vocabulary size; no out-of-vocabulary tokens (with byte-level BPE, literally any input is representable); shares subwords across related words; gives sensible splits for morphology.

Disadvantages. Splits are statistical, not linguistic, so they sometimes cut across morpheme boundaries; token counts are unfair across languages (non-English and non-Latin scripts consume more tokens for the same content, making them more expensive and effectively giving

them less context); and it makes character-level tasks (counting letters, rhyming) unnaturally hard for the model.

The family, and who uses what.

Scheme	Used by
BPE / byte-level BPE	GPT-2, GPT-3, RoBERTa
WordPiece	BERT (merges by likelihood gain rather than raw frequency)
SentencePiece	T5, LLaMA (treats input as a raw byte stream, so no pre-tokenization is needed and whitespace is reversible; works for languages without spaces)

The algorithm, in four lines.

1. Split every word into individual characters and append an end-of-word symbol (`</w>`).
2. Count every adjacent pair across the whole corpus.
3. Merge the single most frequent pair into one new symbol.
4. Repeat for a fixed number of merges — **the only hyperparameter**.

3.15 Worked Example: BPE from Start to Finish

Corpus and hyperparameter. Four words with frequencies: `low` ($\times 5$), `lower` ($\times 2$), `newest` ($\times 6$), `widest` ($\times 3$). Number of merge operations: **10**.

Step 0 — initialise. `l o w </w>` `l o w e r </w>` `n e w e s t </w>` `w i d e s t </w>`

Step 1 — count pairs. `e s` = $6 + 3 = 9$; `s t` = $6 + 3 = 9$; `l o` = $5 + 2 = 7$; `o w` = $5 + 2 = 7$; `e w` = 6; and so on. `e s` and `s t` are tied at 9; take `e s`. Merge: `n e w e s t </w>`, `w i d e s t </w>`.

Steps 2–7. Recount after every merge — that is the part people forget.

Step	Most frequent pair	Count	Result
2	<code>e s t</code>	9	<code>newest</code> → <code>n e w e s t </w></code>
3	<code>l o</code>	7	<code>low</code> → <code>l o w </w></code>
4	<code>l o w</code>	7	<code>low</code> → <code>l o w </w></code>
5	<code>n e</code>	6	<code>newest</code> → <code>n e w e s t </w></code>
6	<code>n e w</code>	6	<code>newest</code> → <code>n e w e s t </w></code>
7	<code>n e w e s t</code>	6	<code>newest</code> → <code>n e w e s t </w></code>

Steps 8–10 mop up lower-frequency pairs in `lower` and `widest` (`w i`, `w i d`, `e r`). After all ten merges `lower` settles as `l o w e r </w>`, because `er` never becomes frequent enough to merge into `low`.

The learned vocabulary. `l`, `o`, `w`, `e`, `r`, `n`, `s`, `t`, `i`, `d`, `</w>`, `es`, `est`, `lo`, `low`, `ne`, `new`, `newest`, `wi`, `wid`, ... — note that frequent whole words survive as single tokens while rarer ones stay decomposed.

The payoff: an unseen word. `lowest` never appeared in training. Apply the learned merge rules *in order*:

`l o w e s t </w> → l o w e s t → l o w e s t → l o w e s t → l o w e s t`.

So `lowest` becomes two known subword units, `low` + `est`. The model never saw the word, but it knows `low` from “low” and “lower” and `est` from “newest” and “widest”, so it can compose a meaning. No `<UNK>` required.

Choosing the number of merges. It is set before training, usually by experimentation, but with guidelines:

- **Morphology:** richer languages (Turkish, Finnish, Arabic) need more merges, because meaning is carried by affixes. English needs fewer.
- **Vocabulary target:** more merges → bigger vocabulary, shorter sequences; fewer merges → more character-level splits, longer sequences.
- **Dataset size:** bigger corpora support more merges, since rarer units still occur often enough to be meaningful.
- **Context length:** longer tokens mean shorter sequences, which matters if your model has a context limit.

The original paper used **32,000–37,000** merges for machine translation, and 32K remains the usual default.

3.16 Transformer: Summary Judgement

Problems it solves. Parallel training; constant-path-length long-range dependencies; a single architecture that transfers to almost every modality.

What it cannot solve.

- Quadratic cost in sequence length — long documents remain expensive.
- It has no memory beyond its context window; it is stateless between calls.
- It is extremely data- and compute-hungry, because it encodes almost no inductive bias (no locality, no recurrence). On small datasets an LSTM or CNN often wins.
- Fixed maximum length: once trained, extending context is non-trivial.

What it gave rise to. The pretraining era (BERT, GPT), and with it a whole new set of problems: models too large to train per task, too large to deploy, misaligned with user intent, and confidently wrong. Parts IV–IX are the responses to those.

Use it when. You have substantial data and GPUs, need long-range dependencies, or want to build on a pretrained checkpoint (which today is nearly always).

Do not use it when. Data is small and no suitable pretrained model exists; sequences are extremely long and you cannot afford $O(n^2)$; or the deployment target is tiny and streaming, where a GRU may be preferable.

4 Part IV: Pretraining, BERT and GPT

4.1 Pretraining and Transfer Learning

The motivating problem. Training a Transformer from scratch is exceptionally difficult: not enough hardware, not enough time, and rarely enough labelled data for the specific task you care about.

The insight. Most of what a model needs to know — syntax, word senses, world facts, discourse conventions — is task-independent. So let an organisation with the hardware train a general *language representation* once on unlabelled text, and let everyone else adapt that representation to their task with a small labelled dataset.

Two ways to use a pretrained model.

- **Fine-tuning:** add a thin task-specific output layer and continue training all (or most) parameters on your labelled data. Usually more accurate.
- **Feature extraction:** freeze the model, run your data through it once, and use the resulting contextual vectors as inputs to any classifier. Cheaper, and lets you precompute embeddings; useful when you cannot afford to fine-tune or must serve many tasks from one frozen backbone. For BERT, concatenating the last four layers was found to be the best feature-based configuration.

Why unlabelled text is enough. Because the training signal is manufactured from the text itself — predict a hidden word, or predict the next word. This is *self-supervised* learning, and it is why the internet becomes a training set.

4.2 BERT

What it is. Bidirectional Encoder Representations from Transformers: a stack of Transformer *encoders* pretrained to produce deep bidirectional representations of text by jointly conditioning on both left and right context in all layers. A pretrained BERT can be fine-tuned with just one additional output layer to reach state of the art on a wide range of tasks.

The idea, plainly. Earlier models read a sentence in one direction (or ran two independent one-directional models and glued them together). BERT wanted a representation of “bank” that is informed by “river” on the left and “flooded” on the right *simultaneously, in every layer*. The obstacle described in the bidirectional RNN section applies: a model that can see the whole sentence cannot be trained to predict the next word, because it would see the answer. BERT’s solution is to change the task.

Sizes.

Model	Encoder layers	Heads / hidden units	Parameters
BERT-base	12	12 heads, 768 hidden	110 million
BERT-large	24	16 heads, 1024 hidden	340 million

Tokenization is WordPiece. Inputs are built as [CLS] sentence A [SEP] sentence B [SEP], with token, segment and position embeddings summed. The final hidden state of [CLS] is the aggregate sequence representation used for classification.

Details the slide leaves out. The feed-forward (filter) size inside each block is always $4H$ — 3072 for BASE, 4096 for LARGE. The per-head dimension is H/A , which is $768/12 = 64$ for BASE and $1024/16 = 64$ for LARGE: *both sizes use the same 64-wide heads*, just more of them. The WordPiece vocabulary has 30,000 entries, so the input embedding matrix alone is $30,000 \times 768 \approx 23.0\text{M}$ parameters for BASE — about a fifth of the whole 110M budget before a single encoder block is counted.

Why BERT-base is exactly GPT-1-sized. $L = 12$, $H = 768$ was chosen deliberately to match GPT-1 so the two could be compared fairly. The architectures are the same size; what differs is the attention pattern (bidirectional versus causal).

4.3 BERT’s Three Input Embeddings

Every input vector is the *sum* of three embeddings of the same width:

$$\text{input}_i = E_{\text{token}}(w_i) + E_{\text{pos}}(i) + E_{\text{segment}}.$$

- **Token embedding** — the ordinary vocabulary lookup.
- **Position embedding** — a *learned* vector per position index (BERT does not use sinuoids).
- **Segment embedding** — says which of the two packed sentences this token belongs to. Only two exist, E_A and E_B , shared by every token on their side. For single-sentence tasks everything gets E_A and the embedding degenerates to a constant.

Why segment embeddings are needed. Self-attention has no inherent notion of sentence boundaries beyond the [SEP] token itself. The segment embedding gives every token an explicit learned signal of which side it is on, which matters directly for NSP and later for question answering (segment A = question, segment B = passage).

Worked example. Sequence [CLS] the cat [SEP] sat down [SEP] with $d_{\text{model}} = 4$. Toy values: $E_{\text{tok}}(\text{cat}) = [0.2, 0.9, 0.1, 0.0]$, $E_{\text{pos}}(2) = [0.0, 0.2, 0.0, 0.1]$, $E_A = [0.05, 0.05, 0.05, 0.05]$, $E_B = [0.15, 0.15, 0.15, 0.15]$. Then

$$[0.2, 0.9, 0.1, 0.0] + [0.0, 0.2, 0.0, 0.1] + [0.05, 0.05, 0.05, 0.05] = [0.25, 1.15, 0.15, 0.15]$$

for “cat” at position 2 in segment A.

The illustrative punchline. The two [SEP] tokens have *identical* token embeddings — both are $[0.0, 0.0, 0.5, 0.5]$ — yet they end up different:

$$\text{pos 3, segment A} \rightarrow [0.25, 0.05, 0.55, 0.75], \quad \text{pos 6, segment B} \rightarrow [0.15, 0.35, 0.65, 0.65].$$

Same token, different final representation — which is exactly what lets the model tell “this separator ends sentence A” from “this one ends sentence B”, purely from the summed embedding, before any attention layer runs.

4.4 Keeping Track of BERT’s Dimensions

768 is BERT-base only. BERT-large uses $H = 1024$. Worked examples usually use 768 because base is the more commonly used variant.

The input is a matrix, not one long vector. A frequent confusion: with 5 tokens and $H = 768$, is the input a 3840-dimensional vector? No — it is a 5×768 *matrix*, one row per token. Sequence length and hidden size are two independent axes, not multiplied together. This matters for attention: self-attention must compare each token against every other individually via dot products, which only makes sense if each token still owns a separate row. Flatten everything and the token boundaries are gone.

768 in, 768 out, at every single layer. Every encoder layer preserves the width:

- **Attention:** Q, K, V are split into 12 heads of $768/12 = 64$; each head outputs 64; concatenating gives $12 \times 64 = 768$; W^O (768×768) keeps it at 768; residual + LayerNorm keeps it at 768.
- **FFN:** $768 \rightarrow 3072 \rightarrow 768$. It expands internally by a factor of 4 with GELU in between, then projects straight back down.

Why it *must* stay constant. The residual connection computes $x + \text{Sublayer}(x)$, and you cannot add two vectors of different sizes. That single constraint forces every layer’s output — across all 12 or 24 blocks — to keep the same width. Dimension changes only *inside* the FFN, never at the interface between layers.

And if you concatenate the last four hidden layers? Each token’s vector grows fourfold: $4 \times 768 = 3072$. For a 5-token sequence the result is 5×3072 . The number of tokens is unaffected; only the per-token width changes.

Pretraining task 1: Masked Language Modelling (MLM). Choose 15% of tokens at random as prediction targets. For each chosen token:

- 80% of the time, replace it with [MASK];
- 10% of the time, replace it with a random token;
- 10% of the time, leave it unchanged.

The model must predict the original token. Because it does not know which positions were corrupted or left alone, it is forced to build a good contextual representation of *every* token.

Why the 80/10/10 split? [MASK] never appears during fine-tuning, so a model trained on 100% [MASK] would suffer a pretrain/fine-tune mismatch. The random-token 10% forces the model to keep a distributional representation of every input token rather than trusting it blindly; the unchanged 10% biases the representation toward the actual observed word. The paper’s ablation shows fine-tuning is fairly robust to the exact ratios, but the feature-based setting (where representations are frozen and the mismatch cannot be corrected) degrades noticeably with a 100% [MASK] strategy. That is the sharpest way to state the justification.

Pretraining task 2: Next Sentence Prediction (NSP). Given sentences A and B, predict whether B actually followed A. Half the training pairs are genuine successors and half are random sentences from the corpus. Motivation: important tasks — question answering, natural language inference — are about relationships *between* sentences, which single-sentence language modelling never teaches. The paper reports NSP reaches 97–98% accuracy on its own — which, in hindsight, is part of the criticism: a task that easy teaches little.

Worked example: one MLM prediction. Toy vocabulary {the, cat, sat, on, mat}, $d = 4$. Sequence “the cat sat” with position 3 masked, so the input is “the cat [MASK]”. Suppose the encoder’s output there is $T_3 = [0.1, -0.2, 0.4, 0.3]$. Project through the output matrix (usually the transposed embedding matrix — weight tying again) to get logits over the vocabulary:

$$\text{logits} = [1.2, 0.3, 2.1, 0.5, -0.4].$$

Softmax, subtracting the max 2.1 for numerical stability:

$$[-0.9, -1.8, 0, -1.6, -2.5] \xrightarrow{\text{exp}} [0.407, 0.165, 1.000, 0.202, 0.082], \quad \text{sum} = 1.856,$$

$$P = [0.219, 0.089, \mathbf{0.539}, 0.109, 0.044], \quad L = -\ln(0.539) \approx 0.619.$$

The true token “sat” (index 2) got probability 0.539. Only this masked position contributes to the MLM loss; the unmasked positions contribute nothing to it.

What “trained jointly” means, mechanically. It does not mean “train MLM to convergence, then train NSP”. For every single training example:

1. One packed input [CLS] A [SEP] B [SEP], with $\sim 15\%$ of tokens masked and a known IsNext/NotNext label.
2. **One forward pass** through the shared encoder.
3. **Two heads read different positions of the same output:** the MLM head reads the masked positions and softmaxes over the vocabulary; the NSP head reads only the [CLS] position and softmaxes over 2 classes.
4. **One combined loss:** $\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$.
5. **One backward pass:** gradients from both losses converge into the same shared encoder weights.

So MLM operates at the *token* level (one loss term per masked token, using each masked position’s own vector T_i) while NSP operates at the *sentence-pair* level (one loss term per example, using only C). Both share the same parameters on every batch.

The pretraining data, precisely. Every training example was a *pair* of text spans, 50% genuine successors and 50% random. Two nuances: a “sentence” here means an arbitrary contiguous span of text, not a grammatical sentence; and A and B together were capped at 512 tokens, so one or both sides were trimmed to fit. Corpus: BooksCorpus ($\sim 800\text{M}$ words) and English Wikipedia ($\sim 2,500\text{M}$ words, text passages only — tables, lists and headers stripped).

Feature extraction: which layer’s vectors should you use? A 12-layer BERT gives 13 vectors per token (the input embedding plus one per block), so you must choose. The paper’s CoNLL-2003 NER ablation:

Feature choice	Dev F1
First layer (embeddings) only	91.0
Last hidden layer only	94.9
Weighted sum of all 12 layers	95.5
Second-to-last hidden layer	95.6
Weighted sum of last four hidden layers	95.9
Concatenation of last four hidden layers	96.1

Summing four 768-vectors keeps the width at 768; concatenating gives $4 \times 768 = 3072$, so more information survives — presumably why it scores best, only 0.3 F1 behind full fine-tuning (96.4). In this setting the frozen features were fed to a freshly initialised two-layer 768-dimensional BiLSTM plus a classification layer; *that* is what gets trained, not BERT.

Advantages.

- Deeply bidirectional context in every layer; excellent for understanding tasks.
- One pretrained model, many downstream tasks, with only a small head added.
- Dramatically reduces labelled-data requirements.

Disadvantages.

- **Cannot generate text.** It is an encoder; there is no autoregressive decoder.
- Only 15% of tokens contribute to the loss per batch, so pretraining converges more slowly per step than left-to-right modelling. (The paper’s ablation confirms MLM converges slightly slower but overtakes left-to-right in absolute accuracy almost immediately.)
- Pretrain/fine-tune mismatch from the artificial [MASK] symbol.
- Requires a fine-tuning step and labelled data per task — no zero-shot ability.
- Fixed 512-token limit; large, slow to serve.

What it cannot solve. Open-ended generation, translation, summarisation, dialogue — anything that produces a sequence.

What it gave rise to. RoBERTa (drop NSP, train longer on more data — showing NSP was not essential), ALBERT (parameter sharing), DistilBERT (distillation), ELECTRA (replaced-token detection to use 100% of tokens as signal), and the general recognition that pretraining objectives are a design space in their own right.

Use it when. Classification, sentiment, NER, extractive QA, sentence similarity, retrieval embeddings — any task where you need to *understand* a fixed text and where a 110M-parameter model that runs cheaply beats calling an LLM API.

Do not use it when. You need generation, or zero/few-shot behaviour with no labelled data, or inputs longer than 512 tokens.

Counterfactual 1. *What if BERT had been trained left-to-right instead of with masking?* It would be GPT. You would gain generation ability and lose the right-context information that makes it so strong at classification and extractive QA. The paper’s own ablation shows the left-to-right variant is worse on MNLI.

Counterfactual 2. *What if masking used 100% [MASK]?* See above: fine-tuning barely changes, but frozen feature quality drops (NER feature-based falls from 94.9 to 94.0 in the ablation), because the model never learns useful representations for *uncorrupted* tokens — which are the only kind it will ever see downstream.

Counterfactual 3. *What if we masked 50% of tokens instead of 15%?* Too little surviving context to reconstruct from, so representations would be poorer. Mask too few, and each expensive forward pass yields almost no training signal. 15% is a compromise between signal per step and context per prediction.

Counterfactual 4. *What if NSP were dropped?* RoBERTa did exactly this and performed *better*, given more data and longer training. The lesson: NSP is a weak objective, partly because “random sentence from a different document” can be detected on topic alone rather than by genuine discourse reasoning.

4.5 GPT: Decoder-Only Generative Models

What it is. A stack of Transformer *decoder* blocks (masked self-attention plus feed-forward; no cross-attention, since there is no encoder) trained with causal language modelling: predict the next token.

The idea, plainly. Nearly every NLP task can be written as text continuation. “Translate to French: I am hungry →” is a prompt; the translation is simply the most likely continuation. So instead of designing one architecture per task, train one model to continue text extremely well and phrase every task as a continuation.

Why decoder-only. Autoregressive next-token prediction requires that position i cannot see position $i + 1$; that is exactly what the decoder’s masked self-attention provides. The encoder–decoder split is only needed when input and output are genuinely different sequences; if you concatenate prompt and answer into one stream, a single masked stack suffices.

What exactly got removed. The original decoder block has *three* sub-layers: masked self-attention, cross-attention, feed-forward. With no encoder to attend to, the cross-attention sub-layer is simply dropped. So every attention operation anywhere in a GPT is masked self-attention — there is no cross-attention at all. Per block: masked multi-head self-attention → add → LayerNorm → feed-forward → add → LayerNorm, repeated 12 times in GPT-1. Note the ordering: GPT-1 uses post-norm, like the original paper.

The objective, written as a factorisation. Concatenate input and target into one sequence and factorise the joint probability into next-token conditionals:

$$p(w_1, \dots, w_{n+\eta}) = \prod_{j=1}^{n+\eta} p(w_j \mid w_1, \dots, w_{j-1}).$$

This single line is the mathematical core of every decoder-only language model.

Two heads at the top. The **text prediction** head is the language-model head used in pretraining: project the hidden state to the vocabulary and softmax. The **task classifier** head is added during fine-tuning and reads the special delimiter token’s representation. During pretraining only the first is active.

Contrast with MLM, in one sentence. BERT predicts only the $\sim 15\%$ of positions it masked, using bidirectional context; a causal LM predicts *every* position’s next token using only leftward context — a far denser training signal per token processed, which is one practical reason decoder-only models became the standard recipe for generation.

4.6 Worked Example: The Language-Model Head

Setup. Let $d_{\text{model}} = 4$ with a vocabulary of five words: cat, dog, sat, mat, the. The final hidden state is $h = [0.5, -0.2, 0.8, 0.1]$, and the embedding matrix W_E has one row per token:

Token	dim1	dim2	dim3	dim4
cat	1.0	0.0	0.5	0.2
dog	0.0	1.0	0.3	-0.1
sat	0.5	-0.5	1.0	0.0
mat	0.2	0.1	0.8	0.3
the	-0.3	0.2	0.1	0.5

Projection: $\text{logit}_v = h \cdot W_E[v]$.

$$\begin{aligned} \text{logit}_{\text{cat}} &= (0.5)(1.0) + (-0.2)(0.0) + (0.8)(0.5) + (0.1)(0.2) = 0.92 \\ \text{logit}_{\text{dog}} &= 0.03, \quad \text{logit}_{\text{sat}} = 1.15, \quad \text{logit}_{\text{mat}} = 0.75, \quad \text{logit}_{\text{the}} = -0.06 \end{aligned}$$

Softmax (with $\sum e^{\text{logit}} = 9.756$) gives $[0.257, 0.106, \mathbf{0.324}, 0.217, 0.097]$, so “sat” wins.

The intuition behind the dot product. A high logit means h points in nearly the same direction as that token’s embedding row. The hidden state encodes “what should come next”; the dot product measures how well each vocabulary embedding matches that signal. With real numbers ($d_{\text{model}} = 768$, $|V| = 50,257$ for GPT-2) this is one $768 \times 50,257$ matrix multiplication scoring every word at once.

4.7 GPT-1: One Model, Many Tasks, via Input Reformatting

GPT-1 adapts to a downstream task *without changing the architecture at all*. Only the input formatting changes, using three special tokens — **Start**, **Delim**, **Extract** — plus one new linear layer reading the **Extract** position’s hidden state.

Task	Input construction
Classification	Start Text Extract $\rightarrow$ Transformer $\rightarrow$ Linear
Entailment	Start Premise Delim Hypothesis Extract $\rightarrow$ Transformer $\rightarrow$ Linear
Similarity	Run <i>both</i> orderings (Text1, Text2) and (Text2, Text1) through the same Transformer, <i>add</i> the two representations, then one Linear. The addition is necessary because the model is causal and therefore order-sensitive, whereas similarity should not depend on which text is listed first.
Multiple choice	One full sequence per candidate (Start Context Delim Answer _{<i>i</i>} Extract), each scored by the same Transformer + Linear, then a softmax across all candidate scores.

The Transformer weights are shared across all tasks; only the linear head is task-specific. This is the same “one additional output layer” pattern as BERT — pretrain once expensively, adapt cheaply.

The generations.

Model	What changed
GPT-1 (2018)	“Improving Language Understanding by Generative Pre-Training.” Established the recipe: generative pretraining on unlabelled text, then discriminative fine-tuning per task. About 120 million parameters.
GPT-2 (2019)	A direct scale-up of GPT-1 — roughly ten times the parameters (1.5 billion) and ten times the data. Trained on WebText: pages linked from Reddit posts with at least 3 karma before December 2017, about 45 million links and 40 GB of text. The conceptual leap: if a language model is powerful enough and the corpus diverse enough, the corpus itself contains demonstrations of many tasks, so <i>fine-tuning becomes unnecessary</i> . Hence the title: “Language Models are Unsupervised Multitask Learners.” Last GPT model released publicly.
GPT-3 (2020)	Scaled again to 175 billion parameters; roughly 300 billion training tokens drawn from CommonCrawl, WebText, English Wikipedia and two book corpora (about 570 GB of plaintext). Reported cost on the order of 355 GPU-years and several million dollars. Its defining property is being a much better <i>in-context learner</i> . API access only. InstructGPT and GPT-3.5 added instruction tuning and RLHF, and made the current LLM boom possible.
GPT-4 (2023)	Multimodality, heavy instruction tuning, reinforcement learning from human and AI feedback, larger context windows (up to 32k tokens in the released variants) and much stronger chat behaviour.

The trajectory in one sentence. GPT-1 = pretrain then fine-tune; GPT-2 = maybe you do not need to fine-tune; GPT-3 = you definitely do not, just show examples in the prompt; GPT-4 = now make it obedient, safe and multimodal.

Advantages. Generation; one model for all tasks; zero- and few-shot ability; capabilities emerge with scale that were not explicitly trained for.

Disadvantages. Unidirectional context, so per-parameter it is weaker than BERT at pure understanding tasks; extremely expensive to train and serve; hallucinates confidently; outputs are stochastic and hard to guarantee.

What it cannot solve. Guaranteed factual accuracy; knowledge after its training cut-off; access to private data; genuine multi-step deliberation in a single forward pass; arithmetic reliability.

What it gave rise to. The whole second half of this syllabus: efficiency methods (the models are too big), alignment methods (they are not obedient), RAG (they are not grounded), agents (they are passive), and evaluation research (nobody knows how to score them).

Use it when. Generation, open-ended tasks, few-shot settings, or when you need one system to cover many tasks.

Do not use it when. You have a narrow, high-volume classification task with labelled data — fine-tuned BERT will be cheaper, faster and often more accurate — or when outputs must be verifiable and deterministic.

Counterfactual. *What if GPT had used an encoder-decoder instead?* You get T5/BART. These are genuinely better for tasks with a clear input-output split (translation, summarisation) because the encoder can be bidirectional over the input. But they are more complex, and once prompting proved that everything can be expressed as continuation, the simpler decoder-only stack won on scaling convenience.

4.8 Choosing an Architecture: The Decision Table

Family	Attention pattern	Best suited to
Encoder-only (BERT, RoBERTa)	Bidirectional	Classification, NER, extractive QA, sentence similarity, embeddings for retrieval
Decoder-only (GPT, LLaMA)	Causal (masked)	Free-form generation, chat, code, few-shot tasks, anything open-ended
Encoder-decoder (original Transformer, T5, BART)	Bidirectional encoder + causal decoder with cross-attention	Translation, summarisation, and other tasks with a distinct source and target

Rule of thumb for the exam. *Understand a fixed text* → encoder. *Produce free text* → decoder. *Transform one text into another* → encoder-decoder.

5 Part V: How Large Language Models Are Pretrained

5.1 The Next-Token Prediction Objective

The task. Given "The capital of Bangladesh is", produce "Dhaka". Formally, maximise the probability the model assigns to the correct next token, over the whole corpus.

The loss. Cross-entropy between the model's predicted distribution over the vocabulary and the one-hot target. For a single position with true token y :

$$\mathcal{L} = -\log p_{\theta}(y \mid x_{<t}).$$

Perplexity, the standard reporting metric, is exp of the average of this loss.

Teacher forcing. During training the model is fed the *ground-truth* previous tokens, never its own predictions. This is what allows every position in a sequence to be trained in parallel in a single forward pass — if the model had to consume its own outputs during training, training would become sequential and lose the Transformer's main advantage.

Autoregressive generation. At inference there is no ground truth, so the model’s own output at step t becomes its input at step $t + 1$.

The tension between them (exposure bias). The model is only ever trained on perfect prefixes but is deployed on prefixes it wrote itself, which contain its own mistakes. Once it makes an error it is in a state it never saw during training, and errors compound. This is one root cause of degeneration and of hallucination cascades.

Counterfactual. *What if we trained without teacher forcing, always feeding the model’s own outputs?* Training would match inference (removing exposure bias), but early in training the model’s outputs are noise, so it would have almost nothing to learn from; convergence would be far slower or fail entirely; and training would become sequential. Scheduled sampling — gradually mixing in the model’s own predictions — is the compromise, but teacher forcing remains standard because parallelism matters more.

5.2 Training Data

Common sources.

- **Common Crawl** — petabyte-scale web pages. Huge but very noisy.
- **Books** — Project Gutenberg and other, less clearly licensed libraries. Long, coherent, high-quality prose; teaches long-range structure.
- **Wikipedia** — clean, structured, factual.
- **Academic papers** — ArXiv, PubMed. Technical vocabulary and reasoning.
- **Code repositories** — GitHub. Notably improves reasoning and structured output even for non-code tasks.

The governing principle. Data quality outweighs raw quantity. Aggressive filtering, deduplication and quality classifiers routinely beat simply adding more raw web text.

Problems with the data.

- **Quality:** spam, machine-generated text, boilerplate.
- **Diversity:** heavy skew toward English and toward the demographics that write most on the web; low-resource languages are badly underserved.
- **Contamination:** benchmark test sets appear in the crawl, so evaluation scores become unreliable (see Section 10).
- **Bias and toxicity:** the model inherits whatever the corpus contains.
- **Legal and ethical status:** copyright, licensing, consent, and personal data scraped without permission.

5.3 Scaling Laws and the Chinchilla Result

What scaling laws are. Empirical power-law relationships showing that loss falls predictably as model parameters, dataset size and compute increase. They let you forecast the performance of a model you have not yet trained, which is what makes multi-million-dollar training runs financially defensible.

The Chinchilla correction (Hoffmann et al., 2022). Before this paper, the field scaled parameters far faster than data. The finding:

- Most existing models were **compute-inefficient**: too large for the amount of data they were trained on.
- Model size and dataset size should be scaled **equally**. Given a fixed compute budget, performance improves more by balancing the two than by inflating the model.
- **Doubling the training data gives comparable or better gains than doubling model size.**
- **Overparameterised** models — too many parameters relative to their training data — underperform smaller models trained on more data.

The evidence. Chinchilla, at 70 billion parameters, outperformed Gopher, a 280-billion-parameter predecessor, using the same compute budget spent on more tokens. The practical heuristic that followed is roughly 20 training tokens per parameter.

The prior belief this overturned. Following Kaplan et al. (2020), the field believed that extra compute should go mostly into making the model *larger*, with training tokens a secondary concern. Chinchilla showed that recommendation dramatically underweighted data. An over-parameterised model has the capacity to learn more but no signal left to learn from.

Where the trade-off comes from: $C \approx 6ND$. Training compute in FLOPs is approximately

$$C \approx 6ND,$$

with N the parameter count and D the number of training tokens. (Roughly 2 FLOPs per parameter per token in the forward pass, times about 3 for forward-plus-backward.) For a *fixed* budget C this ties N and D together: a bigger model means fewer tokens, more tokens means a smaller model. The empirical finding is that the optimum scales the two in equal proportion.

Numerical sanity check. Gopher: $N = 280\text{B}$, $D = 300\text{B}$.

$$C_{\text{Gopher}} \approx 6 \times (280 \times 10^9) \times (300 \times 10^9) = 5.04 \times 10^{23} \text{ FLOPs},$$

close to the paper’s reported 5.76×10^{23} (the gap is expected: the figures are rounded and $6ND$ is an approximation). Now hold that budget fixed and ask how many tokens a 70B model could afford:

$$D = \frac{C}{6N} = \frac{5.76 \times 10^{23}}{6 \times (70 \times 10^9)} \approx 1.37 \times 10^{12} \approx 1.4 \text{ trillion tokens},$$

which is almost exactly Chinchilla’s actual token count. The “4× smaller model, 4× more data, same compute” story is internally consistent, and you can reproduce it in the exam from one formula.

Why this matters beyond training. A smaller model trained on more data is cheaper to *serve*, forever. Training happens once; inference happens billions of times. This is the economic reason the industry moved toward smaller, longer-trained models — and directly why LLaMA (7B/13B trained on trillions of tokens) was possible.

What scaling laws cannot tell you. Which capabilities will appear and when; whether the model will be safe, honest or useful; or what happens when high-quality human text runs out.

Counterfactual. *What if the field had kept scaling parameters only, as before Chinchilla?* We would have a small number of enormous, undertrained, extremely expensive-to-serve models, and no 7B-parameter models good enough to run on a laptop. The entire open-weights ecosystem depends on this result.

6 Part VI: LLaMA and Modern Architectural Refinements

6.1 The LLaMA Family

What it is. A family of decoder-only LLMs published by Meta AI — arguably the most popular freely available LLM — spanning several generations and parameter sizes (7B, 13B, 34B, 70B and others).

Why it matters. It demonstrated the Chinchilla lesson in public: comparatively small models trained on very large token counts, released with open weights, so that academics and small companies could fine-tune state-of-the-art models themselves. It created the open LLM ecosystem.

The generations, in one table.

Model	Data	Params	Context	GQA	Tokens	LR
LLaMA 1	Touvron et al. 2023	7B	2k	×	1.0T	3.0×10^{-4}
		13B	2k	×	1.0T	3.0×10^{-4}
		33B	2k	×	1.4T	1.5×10^{-4}
		65B	2k	×	1.4T	1.5×10^{-4}
LLaMA 2	New public mix	7B	4k	×	2.0T	3.0×10^{-4}
		13B	4k	×	2.0T	3.0×10^{-4}
		34B	4k	✓	2.0T	1.5×10^{-4}
		70B	4k	✓	2.0T	1.5×10^{-4}

Two patterns to notice. (1) The learning rate *decreases* as model size increases within a generation — larger models need smaller updates to train stably. (2) LLaMA 2 doubles the context window (2k→4k) and roughly doubles training tokens. Also note the GQA column: it is × for every LLaMA 1 size and for LLaMA 2 7B/13B, and ✓ only for LLaMA 2 34B and 70B. This table is the precise source that contradicts the slide labelling GQA a “LLaMA 1 new concept”.

Also worth knowing. LLaMA uses SentencePiece tokenisation, and the stack ends with a final RMSNorm before the linear and softmax layers — reading the diagram bottom to top: embeddings; RMSNorm; attention with RoPE applied inside; residual add; RMSNorm; SwiGLU feed-forward; residual add; repeat N times; final RMSNorm; Linear; Softmax.

Scale, to give a sense of cost. LLaMA 2 70B was among the largest LLMs at release: trained on roughly 10 TB of text, requiring about 6000 GPUs for 12 days, on the order of 10^{24} floating-point operations, at a cost around 2 million US dollars.

Architecture at a glance. LLaMA is the original Transformer decoder with four substitutions. Compared side by side with the 2017 architecture:

Component	Original Transformer	LLaMA
Normalisation	LayerNorm, applied after the residual add (post-norm)	RMSNorm, applied to the sub-layer input (pre-norm)
Position information	Sinusoidal encoding added once to the input embeddings	Rotary positional encoding (RoPE) applied to Q and K inside every attention layer
Attention	Multi-head attention	Grouped-query attention with a KV cache (in the larger LLaMA 2 variants)
Feed-forward	ReLU MLP	SwiGLU gated MLP

6.2 Normalisation, and RMSNorm

Why normalise at all? As signals pass through many layers, their scale drifts; activations can grow or shrink until gradients explode or vanish. Normalisation rescales each layer’s activations into a stable range, which permits higher learning rates, reduces sensitivity to initialisation, smooths the loss landscape, and shortens training time considerably.

LayerNorm. Subtract the mean and divide by the standard deviation across the feature dimension, then apply learned gain γ and bias β :

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

RMSNorm. Drop the mean subtraction and the bias; rescale by the root mean square only:

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}}.$$

Toy example. $x = [3, 4, 0, 0]$ with $d = 4$, $\gamma = [1, 1, 1, 1]$, $\epsilon \approx 0$. The root mean square is $\sqrt{\frac{1}{4}(9 + 16 + 0 + 0)} = \sqrt{6.25} = 2.5$, so $\text{RMSNorm}(x) = [1.2, 1.6, 0, 0]$. Note that nothing was re-centred — the zeros stayed zero, which is precisely the difference from LayerNorm.

Why the simplification works. Empirically, the benefit of LayerNorm comes almost entirely from *rescaling*, not from *recentring*. Removing the mean computation removes one pass over the vector and several operations per token, giving a measurable speed-up (commonly cited around 7–15% of training time) at essentially no quality cost. At LLaMA’s scale, a few percent is worth many GPU-days.

Counterfactual. *What if LLaMA had kept post-norm LayerNorm?* Training very deep stacks would require careful learning-rate warm-up and would be more prone to divergence, and each step would be slightly slower. Pre-norm keeps an unobstructed residual path from input to output, which is what makes 80-layer models trainable.

6.3 Positional Encoding, Revisited: Absolute, Relative and Rotary

Absolute (original Transformer). Add a vector that encodes “I am token 7”. The model must infer relative distance from the difference of two absolute encodings. It also does not extrapolate well past the trained length.

Relative (Shaw et al.). Inject a learned embedding of the *offset* $i - j$ directly into the attention score computation. Linguistically this is the right quantity — what matters is that the adjective is two words before the noun, not that it is at absolute position 41. The drawback is extra computation and memory inside every attention operation. Mechanically, for query position i and key position j :

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + a_{ij}^K)^\top}{\sqrt{d_z}}, \quad \alpha_{ij} = \text{softmax}_j(e_{ij}), \quad z_i = \sum_j \alpha_{ij} (x_j W^V + a_{ij}^V).$$

The learned vectors a_{ij}^K and a_{ij}^V are indexed *only* by the offset $j - i$, not by i and j separately, so every pair of positions with the same gap reuses the same vector. The offset is clipped to some maximum $\pm k$, so only $2k + 1$ vectors are ever learned regardless of sequence length — and, crucially, offsets beyond $\pm k$ cannot be distinguished at all.

Worked example (Shaw). Let $d_z = 2$, sentence “the cat sat”, query at $i = 3$. Keys $k_1 = [1, 0]$, $k_2 = [0, 1]$, $k_3 = [1, 1]$; query $q_3 = [1, 1]$; learned relative-key biases $a_{-2}^K = [0.1, 0.1]$, $a_{-1}^K = [0.2, -0.1]$, $a_0^K = [0, 0]$. Then

$$e_{3,1} = \frac{[1, 1] \cdot [1.1, 0.1]}{\sqrt{2}} = 0.849, \quad e_{3,2} = \frac{[1, 1] \cdot [0.2, 0.9]}{\sqrt{2}} = 0.778,$$

$$e_{3,3} = \frac{[1, 1] \cdot [1, 1]}{\sqrt{2}} = 1.414,$$

and softmax gives $\alpha_{3,\cdot} = [0.271, 0.252, 0.477]$. With values $v_1 = [0.5, 0.5]$, $v_2 = [1, 0]$, $v_3 = [0, 1]$ and value biases $a_{-2}^V = [0, 0.1]$, $a_{-1}^V = [0.1, 0]$, $a_0^V = [0, 0]$:

$$z_3 = 0.271[0.5, 0.6] + 0.252[1.1, 0] + 0.477[0, 1] = [0.413, 0.639].$$

Notice the offset bias enters *twice* — once in the score through the key, once in the output through the value.

Rotary positional encoding (RoPE), used by LLaMA. Do not *add* anything. *Rotate* the query and key vectors by an angle proportional to their position, treating consecutive pairs of dimensions as points in a 2-D plane:

$$q'_m = R_m q_m, \quad k'_n = R_n k_n, \quad q'^\top_m k'_n = q_m^\top R_{n-m} k_n.$$

The crucial property: because rotations compose, the dot product of a rotated query and a rotated key depends only on the *difference* $n - m$. So an absolute-style operation produces relative-position behaviour for free.

The algebra, in two lines. For rotation matrices $R(a)^\top = R(-a)$ and $R(-a)R(b) = R(b-a)$. So

$$q_m \cdot k_n = q^\top R(m\theta)^\top R(n\theta) k = q^\top R((n-m)\theta) k,$$

which depends only on $n - m$. In d dimensions the same 2-D rotation is applied independently to $d/2$ coordinate pairs $(1, 2), (3, 4), \dots$, each with its own angle $m\theta_t$, where the θ_t form a geometrically decreasing sequence — the same frequency-schedule idea as the original sinusoids. In practice it is computed element-wise in $O(d)$ rather than by building the full $d \times d$ matrix.

Numerical walkthrough. Let $\theta = 90^\circ$ and $q = k = [1, 0]$ before rotation.

- $m = 1, n = 3$ (offset 2): $q_1 = R(90^\circ)[1, 0] = [0, 1]$ and $k_3 = R(270^\circ)[1, 0] = [0, -1]$. Dot product = -1 .
- $m = 6, n = 8$ (offset 2 again, but completely different absolute positions): $R(540^\circ) = R(180^\circ)$ so $q_6 = [-1, 0]$; $R(720^\circ) = R(0^\circ) = I$ so $k_8 = [1, 0]$. Dot product = -1 .

The same offset gives the same score even though the absolute positions are unrelated. That is the relative-position property, verified arithmetically.

Shaw versus RoPE, side by side. Shaw *adds* a small *learned* vector, indexed by a *clipped* offset — it costs $2k + 1$ trained parameters and cannot tell apart offsets beyond $\pm k$. RoPE *multiplies* (rotates) Q and K by a fixed, *unlearned*, deterministic function of absolute position; the relative behaviour falls out of the geometry with no extra parameters and no clipping, so it extends to sequence lengths never seen in training, which a clipped learned table simply cannot.

Why it is the modern default.

- Gives relative position information at close to zero extra cost, with no added parameters.
- Attention naturally decays with distance, a useful inductive bias.
- Applied to Q and K at every layer, not just once at the input, so position information does not fade with depth.
- Extends to longer contexts by interpolating or rescaling the rotation frequencies — the standard trick behind long-context fine-tuning.

Counterfactual. *What if RoPE rotated the value vectors as well?* The output would become position-dependent in an uncontrolled way and the clean relative-position identity above would break. Rotation is deliberately confined to the matching operation (Q and K), not the content being retrieved (V).

6.4 Multi-Query and Grouped-Query Attention

The problem it addresses. Modern GPUs compute far faster than they can move data. Arithmetic is cheap; reading memory is expensive. So the bottleneck in *inference* is often memory bandwidth, not FLOPs.

Where the memory goes: the KV cache. During autoregressive generation, the keys and values of all previous tokens are cached so they need not be recomputed for each new token. With multi-head attention, each of h heads stores its own K and V for every position. For a long conversation this cache becomes gigabytes, and it must be read from memory for every single generated token.

The spectrum of solutions.

Scheme	Key/value heads	Effect
Multi-head attention (MHA)	One K/V pair per query head	Best quality, largest cache
Multi-query attention (MQA)	A single K/V pair shared by all query heads	Smallest cache, fastest, some quality loss
Grouped-query attention (GQA)	Query heads are split into groups; one K/V pair per group	Near-MHA quality at close to MQA speed

Queries stay fully multi-headed in all three; only the keys and values are shared.

Why it works. The expressive power of attention lives mostly in having many different *queries* — many different questions being asked. Having many distinct *keys and values* matters less. GQA exploits that asymmetry. Formally, with h query heads: MHA has h K heads and h V heads; GQA splits the h query heads into $g < h$ groups with g K heads and g V heads, each shared by h/g query heads; MQA is the extreme $g = 1$.

Numerical illustration. Take $h = 8$ query heads with $d_{\text{head}} = 64$ and count cached values *per token*:

Scheme	K cache per token	K+V total per token
MHA	$8 \times 64 = 512$	1024
GQA, $g = 4$	$4 \times 64 = 256$	512 — half of MHA, all 8 query heads intact
MQA, $g = 1$	$1 \times 64 = 64$	128 — an $8\times$ reduction

Everything else — the causal mask, the softmax, the value-weighted sum — is unchanged. The only thing GQA alters is how many independent K/V projections exist relative to Q projections.

Why the KV cache is what blows up. Its size grows as (batch size $\times$ sequence length $\times$ layers $\times$ KV heads $\times$ head dimension). Long conversations and many concurrent users multiply those factors together quickly, which is why a 70B model can exhaust GPU memory even when the *weights* fit comfortably. Note the diagnostic consequence: quantizing only the weights would not help there, because the weights were never the binding constraint.

Important correction. The lecture slides list grouped multi-query attention among “LLaMA 1 new concepts”. In the published papers, LLaMA 1 used standard multi-head attention; GQA was introduced in **LLaMA 2**, and only in the 34B and 70B variants (7B and 13B kept MHA). Know both the slide claim and the correct attribution.

Counterfactual. *What if we shared the queries instead of the keys and values?* All heads would ask the same question of different content, which destroys the entire point of multi-head attention, and it would not shrink the KV cache at all — queries are not cached, since they are recomputed for the current token only.

6.5 SwiGLU

What it is. A gated feed-forward layer that replaces the ReLU MLP:

$$\text{SwiGLU}(x) = (\text{Swish}(xW_1) \odot xW_3)W_2, \quad \text{Swish}(z) = z \cdot \sigma(z).$$

The idea, plainly. A ReLU MLP applies one nonlinearity and passes the result on. SwiGLU computes two projections: one becomes a smooth *gate* and the other the *content*, and they are multiplied elementwise. It is the same gating intuition as the LSTM — let the network decide, per feature and per token, how much signal to let through — applied inside the feed-forward block.

Practical details. It uses three weight matrices instead of two, so to keep the parameter count unchanged the hidden dimension is reduced to about $\frac{2}{3}$ of the usual $4d_{\text{model}}$. Swish is smooth and non-monotonic, unlike ReLU, which avoids the “dead neuron” problem where a unit stuck in the negative region receives zero gradient forever.

Honest caveat. The original paper introducing GLU variants attributes their success to “divine benevolence” — that is, they work reliably in practice but there is no crisp theoretical reason. Saying this in an exam is better than inventing a justification.

6.6 Long Context Models

Definition. The context window (or context length) of an LLM is the amount of text, in tokens, that the model can consider or “remember” at any one time.

The trajectory. 4K (GPT-3) $\rightarrow$ 128K (GPT-4 Turbo) $\rightarrow$ 1M+ (Gemini 1.5).

Why it matters. Whole-book analysis, entire codebases, long legal contracts, extended multi-turn conversations, and RAG pipelines that can afford to retrieve generously.

Limitations.

- **“Lost in the middle.”** Retrieval accuracy is high for material at the beginning and end of the context and drops sharply for material in the middle. A large window does not guarantee the model uses all of it.
- **Quadratic attention cost.** Doubling context quadruples attention compute and memory. FlashAttention reduces the memory cost by never materialising the full $n \times n$ matrix (not covered in this course), but the asymptotic compute cost remains.
- Cost and latency scale with context, so a long prompt is expensive on every call.
- Quality within the window can degrade even when the model does not error.
- The KV cache also grows *linearly* with context, so it comes to dominate serving memory.

Putting a number on the quadratic cost. At $N = 4\text{K}$ tokens the attention score matrix has $4096^2 \approx 16.8$ million entries per head. At $N = 128\text{K}$ — only $32\times$ longer — it has $128,000^2 \approx 16.4$ *billion* entries per head, which is $32^2 = 1024\times$ more. That factor-of-1024 for a factor-of-32 increase in length is the whole problem in one line.

Use it when. The task genuinely requires global reasoning over a document.

Do not use it when. You only need a few relevant passages — RAG is cheaper, more current and more auditable than stuffing everything into context.

6.7 Mixture of Experts (MoE)

Core idea. Replace the single feed-forward block in each layer with many parallel “expert” feed-forward sub-networks, and use a small learned *router* to send each token to only a few of them. Most parameters exist but stay dormant for any given token.

The canonical example. Mixtral 8x7B: 8 experts, each about 7B, totalling 47B parameters — not 56B, because only the feed-forward layers are expert-specific; token embeddings, attention layers and so on are shared across experts. Only 2 experts are active per token, so roughly 13B parameters are used per forward pass. Other examples: DeepSeek-V3 (256 experts, 8 active) and Qwen 2.5-MoE.

The trade it makes. Decouples *capacity* (total parameters, i.e. how much the model can know) from *compute* (active parameters, i.e. how much it costs to run a token). You get the knowledge of a 47B model at roughly the inference cost of a 13B model.

Advantages. Far better quality per FLOP; faster training and inference for a given knowledge capacity; natural specialisation among experts.

Disadvantages.

- **All** parameters must still fit in memory, even the inactive ones — so VRAM requirements follow the 47B, not the 13B.
- Training is unstable: routers collapse toward a few favourite experts unless an auxiliary load-balancing loss is added.
- Complex to serve efficiently, especially across multiple GPUs.
- Fine-tuning is harder and more prone to overfitting.

Use it when. You are compute-bound at inference but have plenty of memory, and you want maximum capability per FLOP.

Do not use it when. Memory is the binding constraint (edge devices), or you need a simple, easily fine-tuned model.

Counterfactual. *What if the router were random instead of learned?* Experts would never specialise; you would effectively be training an ensemble of identical average models and the capacity advantage would vanish. Conversely, *what if we removed the load-balancing loss?* The router would send nearly all tokens to a handful of experts, the rest would be starved of gradient, and the effective model would shrink to those few — “expert collapse”.

7 Part VII: Faster, Cheaper, Deployable Models

The four techniques in this part answer one question: the pretrained model is excellent but too large to fine-tune or serve. What can be removed?

Technique	What it reduces	What stays the same
Quantization	Bits per weight	Architecture, parameter count
PEFT / LoRA	Trainable parameters	Model size at inference
Pruning	Number of weights or structures	Precision of what remains
Distillation	The whole model (new, smaller one)	Nothing — it is a different model

7.1 Quantization

What it is. A technique to reduce the computational and memory cost of inference by representing weights and activations with low-precision data types, such as 8-bit integers (int8), instead of the usual 32-bit floating point (float32).

Why fewer bits helps. The model needs less memory storage; it consumes less energy (in theory); and operations such as matrix multiplication run much faster with integer arithmetic. In practice, memory bandwidth is the main win: a 4-bit model moves one eighth as many bytes per token as a float32 model.

Two regimes.

- **Post-training quantization (PTQ).** Quantize an already-trained model. No retraining, so it is fast and cheap; accuracy may drop.
- **Quantization-aware training (QAT).** Simulate quantization during training — the forward pass uses quantized values while the backward pass keeps full precision. Recovers more accuracy, but requires a training run.

Common data types and accumulation types.

Storage type	Accumulation type
float16	float16
bfloat16	float32
int16	int32
int8	int32

The **accumulation data type** specifies the type of the result of accumulating (adding, multiplying) values of the storage type. Example: two int8 values $A = 127$ and $B = 127$; their sum $C = 254$ far exceeds the largest representable int8 value. Without a wider accumulator, the result would overflow and the precision loss would make the whole quantization pointless.

What the bits are actually doing. Every floating-point format is 1 sign bit + exponent bits + mantissa bits. The exponent bits buy **range** (how large or small a value can be); the mantissa bits buy **precision** (how many significant digits).

Format	Sign	Exponent	Mantissa
FP32	1	8	23
TF32 (used internally in matmul)	1	8	10
FP16	1	5	10
BFLOAT16	1	8	7

Read the table as a single story: FP16 has only 5 exponent bits, so its *range* is narrower than FP32's — its smallest normal positive value is about $2^{-14} \approx 6 \times 10^{-5}$. BFLOAT16 keeps FP32's full 8 exponent bits (same range) and pays for it with only 7 mantissa bits (coarser precision).

That is exactly the trade that makes bfloat16 the safe training format: it gives up digits, not range.

Case 1: float32 $\rightarrow$ float16. Straightforward, because both types follow the same representation scheme. Two checks are required:

- *Does the hardware support it?* Intel CPUs long supported float16 only as a storage type, converting to float32 for computation; full support arrived with Cooper Lake and Sapphire Rapids.
- *Is the operation sensitive to lower precision?* The ϵ in LayerNorm is typically about 10^{-12} , but the smallest representable float16 value is about 6×10^{-5} ; the small value underflows to zero and can produce NaNs. The same risk exists at the top of the range for large values. This is exactly why bfloat16 — which keeps float32’s exponent range and sacrifices mantissa bits instead — became the standard training format.

Case 2: float32 $\rightarrow$ int8. Only 256 distinct values are representable in int8, while float32 spans an enormous range. The task is to project a float range $[a, b]$ onto the int8 grid. The **affine quantization scheme** is

$$x = S(x_q - Z), \quad x_q = \text{round}\left(\frac{x}{S} + Z\right),$$

where x_q is the int8 value corresponding to x ; S is the *scale*, a positive float32; and Z is the *zero-point*, the int8 value that maps exactly to float 0. Representing zero exactly matters because zero appears everywhere in machine learning (padding masks, ReLU outputs, sparse weights); if it were only approximated, error would be injected into every padded position.

Worked example. Suppose a weight tensor’s observed range is $[a, b] = [-2.0, 6.0]$, to be mapped onto the 256 codes $0 \dots 255$.

$$S = \frac{b - a}{255} = \frac{8.0}{255} \approx 0.03137,$$

$$Z = \text{round}(-a/S) = \text{round}(2.0/0.03137) = \text{round}(63.75) = 64.$$

Check that the endpoints land on the ends of the code range:

$$x_q(a) = \text{round}(-63.75 + 64) = 0, \quad x_q(b) = \text{round}(191.25 + 64) = 255. \quad \checkmark$$

Now dequantise and see how much was lost:

$$x_q = 0 \Rightarrow 0.03137(0 - 64) = -2.008 \approx -2.0, \quad x_q = 255 \Rightarrow 0.03137(255 - 64) = 5.992 \approx 6.0,$$

and, critically, $x_q = 64 \Rightarrow 0.03137(64 - 64) = 0$ *exactly*. The small errors at the ends are the lossiness; the exact zero is the reason the zero-point exists.

An honest gap in the lecture material. Neither the slides nor the reference papers explain how gradients are computed *through* the rounding step in quantization-aware training, given that rounding to a fixed grid is not differentiable. (In practice a “straight-through estimator” is used, but since that is not covered in the course, it is better to flag the gap than to invent an answer.)

Advantages. Large reductions in memory and bandwidth; often $2\text{--}4\times$ faster inference; makes large models runnable on consumer hardware; requires no architectural change.

Disadvantages. Some accuracy loss, which grows sharply below about 4 bits; outlier activation values in LLMs are notoriously hard to quantize; hardware support is uneven; and quantized models can be harder to fine-tune further.

Use it when. Deploying under memory or latency constraints, or serving many concurrent users.

Do not use it when. You are still training or exploring; or accuracy is critical and unverified; or your hardware lacks native low-precision kernels, in which case you may get the memory saving without the speed-up.

Counterfactual. *What if we quantized with a scale but no zero-point (symmetric quantization)?* The mapping is simpler and slightly faster, and it works well for weights, which are roughly symmetric about zero. It works badly for post-ReLU activations, which are all non-negative — half the int8 range would be wasted on values that never occur.

7.2 Parameter-Efficient Fine-Tuning (PEFT)

What it is. A family of methods that freeze most of the pretrained model’s parameters and layers, and train only a small number of added task-specific parameters. By training a small set of parameters and preserving most of the pretrained structure, PEFT saves time and computational resources.

Common techniques.

- **Adapters.** Insert small bottleneck feed-forward layers between the existing Transformer layers and train only those. Concretely: down-project $H \rightarrow r$, apply a nonlinearity, up-project $r \rightarrow H$, with $r \ll H$.
- **LoRA (Low-Rank Adaptation).** Do not modify W ; learn a low-rank correction $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $r \ll d$. The forward pass becomes $h = Wx + BAx$. Training a 4096×4096 layer with $r = 8$ means training about 65,000 parameters instead of 16.8 million. In full: $A \in \mathbb{R}^{8 \times 4096}$ is 32,768 parameters, $B \in \mathbb{R}^{4096 \times 8}$ is another 32,768, so 65,536 against 16.78M — roughly 256× **fewer**, while still able to express any rank- ≤ 8 correction to that matrix.
- Prompt/prefix tuning: learn continuous “virtual tokens” prepended to the input.

Why low rank is enough. The hypothesis is that adapting a model to a new task requires only a low-dimensional change — the pretrained model already contains the capability, and fine-tuning mostly re-weights what it already knows.

Advantages (as listed in the course). Increased efficiency; faster time-to-value; no catastrophic forgetting; lower risk of overfitting; lower data demands; more accessible AI; more flexible AI.

Expanding two of these: *no catastrophic forgetting*, because the base weights are frozen and cannot be overwritten; *more flexible*, because LoRA adapters are tiny files (megabytes) that can be swapped at serving time, so one base model can serve dozens of tasks. With LoRA the adapter can also be merged back into W after training, adding zero inference latency.

Disadvantages. Usually slightly below full fine-tuning on tasks requiring large behavioural change; the rank r is another hyperparameter; and unmerged adapters add a small serving overhead. Crucially, PEFT does not make the model *smaller* at inference — the full base model must still be loaded.

What it cannot solve. Teaching the model genuinely new knowledge or a new language it never saw in pretraining; PEFT adapts, it does not extend. It also does not help when the base model is simply too large for your hardware.

Use it when. Adapting a large pretrained model to a task or style with limited data and one GPU; maintaining many task variants of a single base model.

Do not use it when. You need to teach substantial new knowledge (consider continued pretraining or RAG); or your model is small enough to fine-tune fully.

Counterfactual. *What if we set the LoRA rank r equal to the full dimension?* ΔW becomes an unconstrained full-rank matrix and you have simply reimplemented full fine-tuning, with all its memory cost and its risk of catastrophic forgetting. The constraint *is* the method.

7.3 Pruning

The procedure.

1. Train the model.
2. Prune — for example remove 30% of the network.
3. Fine-tune to recover accuracy.

Two kinds.

- **Magnitude (unstructured) pruning:** remove individual weights with the smallest absolute values, on the assumption that small weights contribute least. Produces sparse weight matrices with high theoretical compression, but generic hardware cannot exploit irregular sparsity, so the real speed-up is often disappointing.
- **Structured pruning:** remove whole neurons, attention heads or layers. Lower compression per unit of accuracy lost, but the result is a genuinely smaller dense model, which is better for hardware and gives real speed-ups.

The practical difficulty flagged in the lecture. Recovery fine-tuning is necessary, but PEFT may not be strong enough to repair the damage of removing 30% of the network, while a full fine-tune of an LLM is expensive. Pruning large models is therefore harder than it sounds, and this is one reason distillation is often preferred at LLM scale.

Counterfactual. *What if we pruned without the recovery fine-tune?* Accuracy would fall sharply. Removing weights disturbs the balance of every downstream layer; the remaining weights must be readjusted to compensate. Pruning is best understood as “damage then repair”, not simply “delete”.

7.4 Knowledge Distillation

What it is. Train a small “student” model to mimic a large “teacher” model.

How it works.

1. The teacher LLM generates *soft labels* — full probability distributions — over a dataset.
2. The student is trained to match the teacher’s output distribution, minimising the KL divergence between the two distributions:

$$D_{\text{KL}}(P \parallel Q) = \sum_v P(v) \log \frac{P(v)}{Q(v)},$$

where P is the teacher’s distribution and Q the student’s.

Numerical check on the loss. Toy 3-token vocabulary, teacher $P = [0.7, 0.2, 0.1]$.

- A student that matches exactly, $Q_a = [0.7, 0.2, 0.1]$, gives $D_{\text{KL}} = 0$. Identical distributions, zero divergence — a sanity check that the loss is correctly oriented.
- A weak, nearly uniform student $Q_b = [0.34, 0.33, 0.33]$ gives

$$0.7 \ln \frac{0.7}{0.34} + 0.2 \ln \frac{0.2}{0.33} + 0.1 \ln \frac{0.1}{0.33} \approx 0.505 - 0.100 - 0.119 = 0.286 \text{ nats.}$$

Clearly positive for the worse student, zero for the perfect one.

Why soft labels beat hard labels. A hard label says only “this is a cat”. A soft distribution says “85% cat, 12% dog, 0.01% aeroplane”, which encodes the teacher’s learned similarity structure — often called *dark knowledge*. The student learns not just the answer but the shape of the teacher’s uncertainty, so it learns far more per example than it would from the hard labels alone.

Examples. DistilBERT: 40% smaller, retaining 97% of BERT’s accuracy. TinyLlama (1.1B) trained from LLaMA 2 outputs.

Best for. Creating tiny, fast, specialised models — for instance an on-device chat assistant.

Disadvantages. Requires running the large teacher over a large dataset (expensive); the student inherits the teacher’s biases and errors; the student generally cannot exceed the teacher; and licensing may forbid distilling from a commercial API.

Counterfactual. *What if the student were trained on the teacher’s hard argmax outputs instead of soft distributions?* You would lose the dark knowledge and be back to ordinary supervised training on machine-generated labels. Distillation would still help a little (the teacher’s labels are cleaner than raw web data) but the characteristic efficiency of distillation comes from the soft targets.

8 Part VIII: Instruction Tuning and Alignment

8.1 Pretrained Models Are Not Chatbots

A pretrained LLM is a text *continuer*, not an *assistant*. Ask it “What is the capital of Bangladesh?” and a plausible continuation is another exam question, because that is what

usually follows a question on the web. It is not disobedient; it was never trained to obey. The remaining stages fix that.

Stage	What it teaches	Data required
Pretraining	Language, facts, reasoning patterns	Trillions of unlabelled tokens
Instruction tuning / SFT	Format: how to follow an instruction	Thousands to millions of (instruction, response) pairs
RLHF / DPO	Preference: which of two valid answers humans prefer	Human comparisons of model outputs
Constitutional AI	Principles, applied by the model to itself	A written constitution, plus model-generated critiques

8.2 Instruction Tuning

What it is. Fine-tuning an LLM on a labelled dataset of instructional prompts and their corresponding outputs, so that it responds to task-specific prompts phrased in natural language. It is ordinary supervised learning on (input, output) pairs.

Example.

- Instruction: What is the sentiment of this review?
- Input: The movie was boring and too long.
- Output: Negative

Why it works so well. Training on *many* tasks phrased as instructions teaches the general skill “an instruction is a thing to be obeyed”, which then transfers to instructions never seen in training. FLAN (2021) demonstrated this cross-task generalisation.

Result. The model becomes more user-friendly, more general-purpose, and better aligned with human intent — without changing the architecture at all.

8.3 Alignment through Supervised Fine-Tuning

Alignment, defined. The process of shaping a model’s behaviour to be helpful, harmless and honest according to human intentions and values.

SFT is the easiest alignment method.

1. Collect human demonstrations: (prompt, ideal response) pairs.
2. Fine-tune with standard cross-entropy loss on the responses.
3. Evaluate on held-out prompts.

Its ceiling. SFT teaches *format*, not *preferences*. It can show the model what a good answer looks like, but it cannot express that answer A is 30% better than answer B, nor navigate the helpful-versus-harmless trade-off. It is also limited by the quality of human writers — the model imitates demonstrations rather than exceeding them. And every demonstration must be written by a person, which is expensive.

8.4 Reinforcement Learning from Human Feedback (RLHF)

The pipeline.

1. **Collect human preferences.** Show annotators two model responses, A and B, and ask which is better. Comparison is far easier and more reliable for humans than writing an ideal answer or assigning an absolute score.
2. **Train a reward model** that predicts the human preference score for a response. This turns sparse human judgement into a dense, automatic signal that can be queried millions of times.
3. **Optimise the policy** (the LLM) against the reward model using PPO (Proximal Policy Optimization).

Why reinforcement learning at all? Because “be helpful” has no ground-truth target string to imitate. There is no single correct answer to copy, only better and worse ones — which is exactly the setting RL is designed for.

Assessment. RLHF is expensive but produces the most helpful models (ChatGPT, Claude). However, PPO is notoriously unstable; Direct Preference Optimization (DPO) is more practical, because it optimises the same preference objective directly with a supervised-style loss, dispensing with the separate reward model and the RL loop.

Disadvantages and failure modes. Very expensive human annotation; annotator disagreement and bias baked in; *reward hacking*, where the model finds outputs the reward model scores highly but humans dislike (excessive length, flattery, hedging); and an “alignment tax” — some raw capability is often lost.

Counterfactual. *What if we optimised the reward model without a KL penalty keeping the policy near the SFT model?* The policy would drift far from natural language and exploit the reward model’s blind spots, producing degenerate text that scores highly and reads as nonsense. The KL constraint is what keeps RLHF anchored to fluent output.

8.5 Constitutional AI

What it is. A method developed by Anthropic to train and align models to be helpful, honest and harmless — the three pillars of alignment — without relying on humans to manually review and score thousands of toxic or dangerous outputs.

How it works. The model is given a written “constitution”: a set of principles, rules and guidelines. It then uses that constitution to critique and revise its own outputs. The loop runs: constitution → red-teaming → self-critique → self-revision → supervised or reinforcement learning from the resulting feedback.

Advantages. Dramatically reduces dependency on human-generated data; scales far better; makes the values *explicit and auditable* as a written document rather than implicit in thousands of annotator judgements; and spares human workers from repeated exposure to harmful content.

Disadvantages. Someone must still write the constitution, so the value judgements merely move upstream; the model may misinterpret abstract principles; and self-critique is bounded by the model’s own understanding — it cannot catch a flaw it cannot recognise.

9 Part IX: Using LLMs

9.1 In-Context Learning

What it is. Learning from examples provided in the prompt, with no weight updates. Provide one example (one-shot) or a few (few-shot). Few-shot often beats zero-shot by a significant margin.

Advantages. Zero training cost; instant adaptability; no ML infrastructure needed; you can change the “task” between two consecutive API calls.

Why does it work at all? Nothing is being learned in the usual sense — no parameter changes. The leading explanation is that a sufficiently diverse pretraining corpus contains countless implicit task demonstrations, so the model has learned to recognise and continue patterns; the prompt’s examples *locate* an ability the model already has rather than installing a new one. A related view is that the forward pass through attention performs something functionally similar to a small gradient-descent-like update inside the activations. Treat both as hypotheses, not settled fact.

Disadvantages. Examples consume context on every call, so it is expensive at scale; performance is sensitive to example choice, order and formatting; and it cannot match fine-tuning when thousands of labelled examples exist.

Use it when. Prototyping, few labelled examples, rapidly changing requirements.

Do not use it when. High volume with stable requirements and abundant labels — fine-tune or distil instead, and stop paying for the examples on every request.

9.2 Prompt Design

- **Role prompting:** “You are a professional data scientist...”
- **Structured prompting:** “Step 1: Analyze. Step 2: Propose. Step 3: Conclude.”
- **Delimiters:** use `###` or triple quotes to separate instructions from input. This also mitigates prompt injection, since the model can tell data from commands.
- **Output formatting:** “Respond with JSON: {”answer”: ...}”
- **In-context examples:** one-shot or few-shot.
- **Negative constraints:** “Do not include any conversational filler...”
- **Escape hatch:** explicitly permit “I don’t know”. Without it the model’s training pressure to produce a fluent continuation encourages fabrication; giving it a licensed way to decline reduces hallucination.

9.3 Chain-of-Thought (CoT) Prompting

What it is. Asking the model not only to answer but also to generate a rationale — the reasoning steps by which it arrived at the answer.

Effect. CoT significantly enhances performance on arithmetic, symbolic and other logical reasoning tasks, including in the zero-shot case (“Let’s think step by step”).

Why does it work? Two complementary reasons, and the exam will want both:

1. **Computational.** A Transformer performs a fixed amount of computation per generated token. A hard problem may need more computation than one forward pass provides. Writing intermediate steps lets the model spend many forward passes on one problem — the generated tokens act as an external working memory.
2. **Distributional.** Correct reasoning chains in the training corpus are usually followed by correct answers. Conditioning on a well-formed chain of reasoning shifts the model into a region of its distribution where correct answers are far more likely.

Limitations. It costs many extra tokens (latency and money); the stated rationale is not guaranteed to be the actual cause of the answer, so it is a poor faithfulness guarantee — a model can produce a flawless-looking chain and a wrong answer, or the reverse; it mainly helps large models, and can hurt small ones; and it does not help on tasks with no decomposable structure.

9.4 The Problem with Pure LLMs

- **Hallucinations** — fluent, confident, false statements.
- **Outdated knowledge** — frozen at the training cut-off.
- **No access to private data** — it never saw your company’s documents.
- **Expensive to retrain** — you cannot re-pretrain a model every time a fact changes.

Solution: do not retrain. Give the LLM a retriever.

9.5 Retrieval-Augmented Generation (RAG)

What it is. An architecture combining a **retriever**, which finds relevant documents in a knowledge base, with a **generator**, which produces an answer conditioned on those documents.

The retriever (the “search engine”).

- Convert documents into vector embeddings.
- Store them in a vector database (Pinecone, Weaviate, FAISS).
- At query time, embed the question and find the top- k most similar documents.

Vector search matters because it matches on *meaning*: a query about “annual leave” retrieves a passage about “vacation policy” even with no shared keywords.

The generator (the LLM).

- Takes [retrieved documents] + [user query] + [instructions].
- Generates an answer conditioned only on the retrieved context.

- Can be any LLM. Notably, small LLMs (around 7B) work surprisingly well as generators when retrieval is good — because the hard part becomes reading, not recalling.

Use cases. Open-domain question answering; chatbots that must be factually accurate; enterprise Q&A over internal documents; keeping responses current without retraining.

Advantages. Knowledge updates instantly by editing the document store; sources can be cited, so answers are auditable; private data never enters model weights; and it works with a smaller, cheaper generator.

Disadvantages. The whole system is capped by retrieval quality — if the right passage is not retrieved, the answer will be wrong; added latency and infrastructure; chunking strategy matters a great deal; and the model may still ignore or contradict the retrieved context.

RAG versus fine-tuning.

Dimension	RAG	Fine-tuning
Knowledge freshness	Immediate — update the index	Stale — requires retraining
Transparency	High — the retrieved sources can be shown	Low — knowledge is diffused into weights
Compute cost	Low up-front, higher per query (longer prompts)	High up-front, cheaper per query
Hallucination risk	Lower, because answers are grounded in retrieved text	Unchanged or worse; fine-tuning on facts can teach confident invention
Best at teaching	<i>Knowledge</i> — what to say	<i>Behaviour</i> — how to say it (style, format, tone)

The classwork question, answered. *You need a model that imitates the writing style of a particular newspaper while also being factual — which solution?* Both, and the split follows the last row of the table. Fine-tune (or LoRA) on that newspaper’s archive to acquire its *style*; use RAG to supply the *facts* for each specific article. Style is behaviour, so it belongs in the weights; facts are knowledge, so they belong in the context. Fine-tuning alone would produce a model that writes beautifully in house style and invents its facts.

9.6 Reasoning Models and Agentic AI

Standard LLMs are “System 1” thinkers. Fast, automatic, pattern-matching; one forward pass yields one answer; autoregressive, with no ability to backtrack, revise or plan. Consequently they often fail at multi-step mathematics, logic puzzles, planning and self-correction.

Reasoning LLMs. A reasoning model is an LLM fine-tuned to break complex problems into smaller steps — often called *reasoning traces* — before producing a final output. The distinction to hold onto is *pattern matching versus deliberate reasoning*. CoT prompting asks for this behaviour at inference time; a reasoning model has been trained to do it natively, typically with reinforcement learning against verifiable answers. Reasoning LLMs play a critical role in agentic AI.

The agent loop. At each iteration the agent assembles context from the available inputs, invokes an LLM to reason and select an action, executes that action, observes the outcome, and feeds the observation back into the next iteration. The loop repeats until the task is complete or a stopping condition is reached.

Why the loop is the key idea. It converts a one-shot text generator into a system that can act, observe consequences and correct itself. The model gains a feedback signal from the world, which is exactly what a single forward pass lacks.

Convergence. Every major AI organisation has converged on the same execution pattern, summarised by Lilian Weng as

$$\text{Agent} = \text{LLM} + \text{Memory} + \text{Planning} + \text{Tool Use}.$$

Memory supplies persistence beyond the context window; planning decomposes the goal; tool use provides grounding (search, code execution, databases, APIs). The Model Context Protocol (MCP) standardises how tools are exposed to models, so that any tool can be plugged into any agent without bespoke integration.

Disadvantages. Errors compound across iterations; loops can run away in cost; tool-calling introduces security concerns such as prompt injection; and evaluation is very hard, because success is defined over a whole trajectory rather than a single output.

10 Part X: Evaluation and Responsibility

10.1 Why Evaluating LLMs Is Hard

- Generative models produce open-ended responses, so correctness is often **subjective** — there is no single reference answer to compare against.
- Alignment is subjective as well: how helpful a model is depends on the user.
- **Benchmark contamination:** massively trained LLMs may already have been trained on the very data used to test them, so a high score may measure memorisation rather than ability.

Add to these that capabilities are unevenly distributed (a model can pass a bar exam and fail at counting letters), and that benchmark scores are sensitive to prompt formatting.

10.2 Common Benchmarks

Benchmark	What it measures
MMLU	Multitask accuracy across 57 subject areas
HumanEval	Code generation
BIG-Bench	Diverse reasoning tasks
GSM8K	Grade-school mathematics word problems
MT-Bench	Multi-turn conversation and reasoning

Benchmarks are useful but not sufficient. They are cheap, comparable and reproducible; they are also gameable, contaminated over time, and silent about the qualities users actually care about.

10.3 LLM-as-a-Judge

Extremely popular at present: if prompted properly, a strong LLM scoring another model's output is a scalable, flexible and surprisingly accurate alternative to human evaluation.

Advantages	Disadvantages
Fast and scalable	Verbosity bias — longer answers are scored higher
Aligns reasonably well with human preferences	Egocentric bias — a judge prefers text resembling its own output
Flexible — any criterion can be written into the rubric	Cost — an extra model call per evaluation
Consistent — no annotator fatigue or drift	Ordering bias — position of A and B affects the verdict
Explainable — the judge can state its reasons	Trustworthiness — the stated reasons may not be the real ones

Practical mitigations: swap the order of candidates and average; use a fixed rubric; and calibrate the judge against a human-scored sample.

10.4 Human Evaluation

The gold standard, but expensive. For some qualities — humour, empathy, tone — it may be the only valid method. It is also inconsistent and subjective across annotators, and scaling it is a nightmare. In practice the field uses a pyramid: automatic benchmarks for fast iteration, LLM-as-judge for breadth, human evaluation for final decisions.

10.5 Responsible LLMs

- **Hallucinations.** Fluency is optimised, truth is not. Mitigations: RAG, citations, escape hatches, calibration, human review for high-stakes use.
- **Bias and fairness.** Models reproduce and can amplify the distribution of their training data — gender, race, religion, language and geography. Mitigations: data auditing, targeted evaluation, alignment training. Note that debiasing is itself a value judgement about which distribution is correct.
- **Privacy and intellectual property.** Training data may contain personal information and copyrighted work; models can memorise and regurgitate rare strings; and the legal status of training on scraped text remains contested.
- **Safety and security.** Misuse for disinformation, fraud and malware; jailbreaks and prompt injection; over-reliance by users on confident wrong answers; and, in agentic settings, real actions taken in the world.

10.6 Future Directions

Long-horizon models that maintain coherence over extended tasks; personalised assistants; on-device LLMs (which is what Part VII's efficiency methods are ultimately for); stronger reasoning; autonomous agents; and alignment and interpretability, so that we can verify what these systems are actually doing.

11 Appendix A: Points Where the Slides Need Care

1. **Grouped-query attention is listed under “LLaMA 1 new concepts”**. In the published papers, LLaMA 1 used standard multi-head attention. GQA appeared in **LLaMA 2**, and only in the 34B and 70B variants. If a question asks which LLaMA generation introduced GQA, the correct answer is LLaMA 2.
2. **RMSNorm appears in the LLaMA architecture diagram but not in the bulleted list of new concepts**, where the item is simply “Normalization”. RMSNorm is one of LLaMA’s four defining changes and should be named explicitly alongside RoPE, GQA and SwiGLU.
3. **GPT-1’s parameter count** is given as 120 million in the slides; the paper’s configuration is usually reported as about 117 million. Either figure is acceptable; do not build an argument on the difference.

12 Appendix B: One-Page Comparison Tables

12.1 Recurrent versus Attention-Based

Property	Simple RNN	LSTM / GRU	Transformer
Path length between distant tokens	$O(n)$	$O(n)$	$O(1)$
Parallel over positions	No	No	Yes
Compute per layer	$O(n \cdot d^2)$	$O(n \cdot d^2)$	$O(n^2d + nd^2)$
Long-range dependencies	Poor	Moderate	Strong
Memory of past	One vector	Cell state plus hidden state	Full context, re-read each step
Inductive bias	Strong (recency)	Strong (recency, gated)	Weak — needs much more data
Best regime	Tiny data, streaming	Small–medium data	Large data plus GPUs

12.2 Which Efficiency Method for Which Constraint

Your constraint	Reach for
Model will not fit in VRAM	Quantization (and, if available, a smaller distilled model)
Cannot afford to fine-tune all parameters	PEFT / LoRA
Need many task variants served from one base	LoRA adapters swapped at serving time
Need a genuinely smaller and faster model	Distillation, or structured pruning
Need more knowledge capacity at fixed FLOPs	Mixture of Experts
Need current or private facts	RAG (not fine-tuning)
Need a particular style or format	Fine-tuning (not RAG)

13 Appendix C: Rapid Revision Questions

1. **Why does the LSTM solve the vanishing gradient problem?** Because the cell state update is additive, $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, so the gradient through time is multiplied by the forget gate rather than by a chain of tanh derivatives. With $f_t \approx 1$ the gradient passes back nearly unchanged.

2. **Why divide by $\sqrt{d_k}$ in attention?** Dot products of high-dimensional vectors have large variance; large logits saturate the softmax, whose gradient then vanishes. Scaling keeps logits in a range where softmax remains trainable.
3. **Why does a Transformer need positional encoding when an RNN does not?** Self-attention is permutation-invariant — it treats the input as a set. An RNN encodes order implicitly through the sequential order of its updates.
4. **Why can BERT not generate text?** It is an encoder trained with bidirectional attention. Any position can see every other, so there is no way to define a next-token prediction that does not leak the answer, and there is no autoregressive decoder.
5. **Why does BERT use 80/10/10 masking?** [MASK] never occurs at fine-tuning time, so training on 100% [MASK] creates a pretrain/fine-tune mismatch. Random and unchanged tokens force good representations of ordinary tokens too, which matters most in the frozen feature-extraction setting.
6. **What is the single most important claim of the Chinchilla paper?** For a fixed compute budget, model size and training data should scale equally; most prior models were too large for their data, and doubling data helps at least as much as doubling parameters.
7. **Why did GQA appear?** GPUs compute much faster than they move memory, so autoregressive inference is bandwidth-bound, and the KV cache dominates that bandwidth. Sharing keys and values across groups of query heads shrinks the cache with little quality loss.
8. **Why is RoPE preferred over sinusoidal encoding?** It encodes relative position through rotation, so attention scores depend only on the offset between positions; it is applied inside every layer rather than once at the input; it adds no parameters; and it can be extended to longer contexts by rescaling frequencies.
9. **What does a MoE model buy you?** Knowledge capacity decoupled from per-token compute: many parameters exist, few are active. The cost is that all of them must still be held in memory, plus routing instability.
10. **Difference between quantization, pruning, PEFT and distillation?** Quantization reduces the precision of weights; pruning removes weights or structures; PEFT reduces the number of *trainable* weights without shrinking the model; distillation trains a new, smaller model to imitate a large one.
11. **Why RLHF rather than more supervised fine-tuning?** SFT can only imitate demonstrations, so it teaches format, not preference, and cannot express that one valid answer is better than another. Preference comparisons are easier for humans to give and support optimisation beyond what any demonstrator wrote.
12. **RAG or fine-tuning?** RAG for knowledge that is fresh, private, large or must be cited; fine-tuning for behaviour — style, format, tone, task-specific output structure. Combine them when you need both.
13. **Why does chain-of-thought help?** It gives the model more computation for a hard problem (each generated token is another forward pass, and the text acts as working memory) and conditions it on the kind of context that, in the training distribution, precedes correct answers.
14. **What is the biggest weakness of long-context models?** They do not use the whole window equally — the “lost in the middle” effect — and attention cost grows quadratically with length.
15. **Name the four architectural differences between LLaMA and the original Transformer decoder.** Pre-norm RMSNorm instead of post-norm LayerNorm; rotary positional encoding instead of sinusoidal; grouped-query attention with a KV cache instead of plain multi-head attention (LLaMA 2, larger variants); and a SwiGLU gated feed-forward network instead of a ReLU MLP.